Code Intelligence and Change Impact Analysis Engine

DBMS mini project report. UE23CS351A Database Management Systems, PES University.

1. Title

Code Intelligence and Change Impact Analysis Engine

A web application that reads a Git repository into PostgreSQL, MongoDB and pgvector. From that data it answers questions about the code: what a change affects, who changes what, where code is duplicated, and which function does what a user describes.

2. Students

Section 5 C, UE23CS351A Database Management Systems.

SRN	Name	Area of ownership
PES1UG24CS177	Guru R Bharadwaj	Code structure and impact analysis: `repository`, `file`, `symbol`, `calls` and `imports`, the MongoDB `ast` collection, `impact_of`, symbol search, the span trigger, the parser, and the repository, file, symbol, call graph and impact pages
PES1UG24CS901	Vivian Sobers E	History, similarity and access: `author`, `author_email`, `commit`, `modifies`, `clone_cluster`, `app_user` and `waives`, pgvector embeddings, the history views, the clone review procedure and trigger, authentication, and the dashboard, commit, search, clone review and user pages

The full split, including shared tasks, is in WORK_SPLIT.md.

3. Abstract

Before changing a function in an unfamiliar codebase, a developer needs to know what calls it, directly or through other functions, and which entry points end up running it. They also want to know who changed it recently, how often it changes, and whether the same logic was copied somewhere else. Git and an editor can answer each question, but slowly and one at a time. `git log` lists commits without saying which functions they touched, and text search cannot follow a call through two levels of indirection or find code that does the same thing under another name.

The engine parses JavaScript, TypeScript, Python, Java and Go repositories with tree-sitter. Files, symbols, calls and imports go into PostgreSQL, and each file's syntax tree is stored as a MongoDB document. A history step reads the full `git log` and records, for every commit, which symbols it added or modified and by how many lines. An embedding step turns every function body into a 768-dimension vector with a local code embedding model and stores it with pgvector.

On top of this data the database answers the project's main questions. A recursive CTE inside the stored function `impact_of` walks the call graph backwards to list everything a change can reach. Views over the history show change hotspots, file ownership and author churn. Cosine similarity over the vectors drives semantic search and groups near-duplicate functions into clone clusters, which an admin reviews through a stored procedure. A Next.js app puts all of this behind two roles, Admin and Developer. The middleware, the route handlers, a trigger and PostgreSQL grants all enforce the role rules.

4. Tech stack

4.1 Components and versions

Layer	Technology	Version	Used for
Relational database	PostgreSQL	16 (`pgvector/pgvector:pg16` image)	Code structure, history, clone review, users; views, functions, procedure, triggers, roles
Vector search	pgvector	bundled with the image	`vector(768)` column, HNSW index, cosine k-NN
Document database	MongoDB	7 (`mongo:7` image); MongoDB Atlas works through `MONGODB_URI`	One syntax tree document per file, aggregation pipelines
Web framework	Next.js (App Router)	15.5.28	Server-rendered pages, server actions, route handlers under `/api`
UI library	React	19.2.1	Pages and components
Language	TypeScript	5.9	App and ingest tools
Database drivers	`pg`, `mongodb`	8.23.1, 6.21.0	Connection pool and Mongo client
Parsing	`web-tree-sitter` with `tree-sitter-wasms`	0.22.6, 0.1.13	Parsing five languages without a native build
Embeddings	`@huggingface/transformers` with `jinaai/jina-embeddings-v2-base-code`	4.3.2	Local, CPU-only code embeddings, in the ingest tools and the search page
Passwords	`bcryptjs`	3.0.3	bcrypt hashes with cost 12
Sessions	`jose`	6.2.15	HS256-signed session cookie, checked in middleware
Runtime and tests	Node.js, `node:test`, `tsx`	Node 20 or newer	Unit, integration and API smoke tests
Containers	Docker Compose		Local PostgreSQL and MongoDB

4.2 Why three kinds of storage

Repositories, files, symbols, calls and imports have a fixed shape and many links between them. Every symbol belongs to one file and every call joins two symbols, so PostgreSQL holds them with foreign keys and cascades. One transaction can then replace a repository's whole structure without leaving dangling calls. CHECK constraints catch parser bugs at insert time, such as an inverted line span or a confidence of 1.5. The main question, "what breaks if I change this?", means following `calls` backwards a few levels, which a recursive CTE does with an index lookup per step. We looked at a graph database such as Neo4j. Traversal

syntax would be shorter, but the call graph would then live apart from the commits and clone data it is joined with, and every query would cross two systems. Full-text search over symbol names also runs in PostgreSQL, through a generated `tsvector` column and a GIN index.

A syntax tree has a different shape for every language and every file. Stored relationally it would need a generic node table with tens of thousands of rows per file and a self-join for each level of nesting. No query joins a single AST node to another table: the app reads a file's tree as a unit or aggregates over its nodes. MongoDB stores one document per file, fetched by key, with aggregation pipelines for the statistics. The relational side keeps only `file.ast_doc_id`.

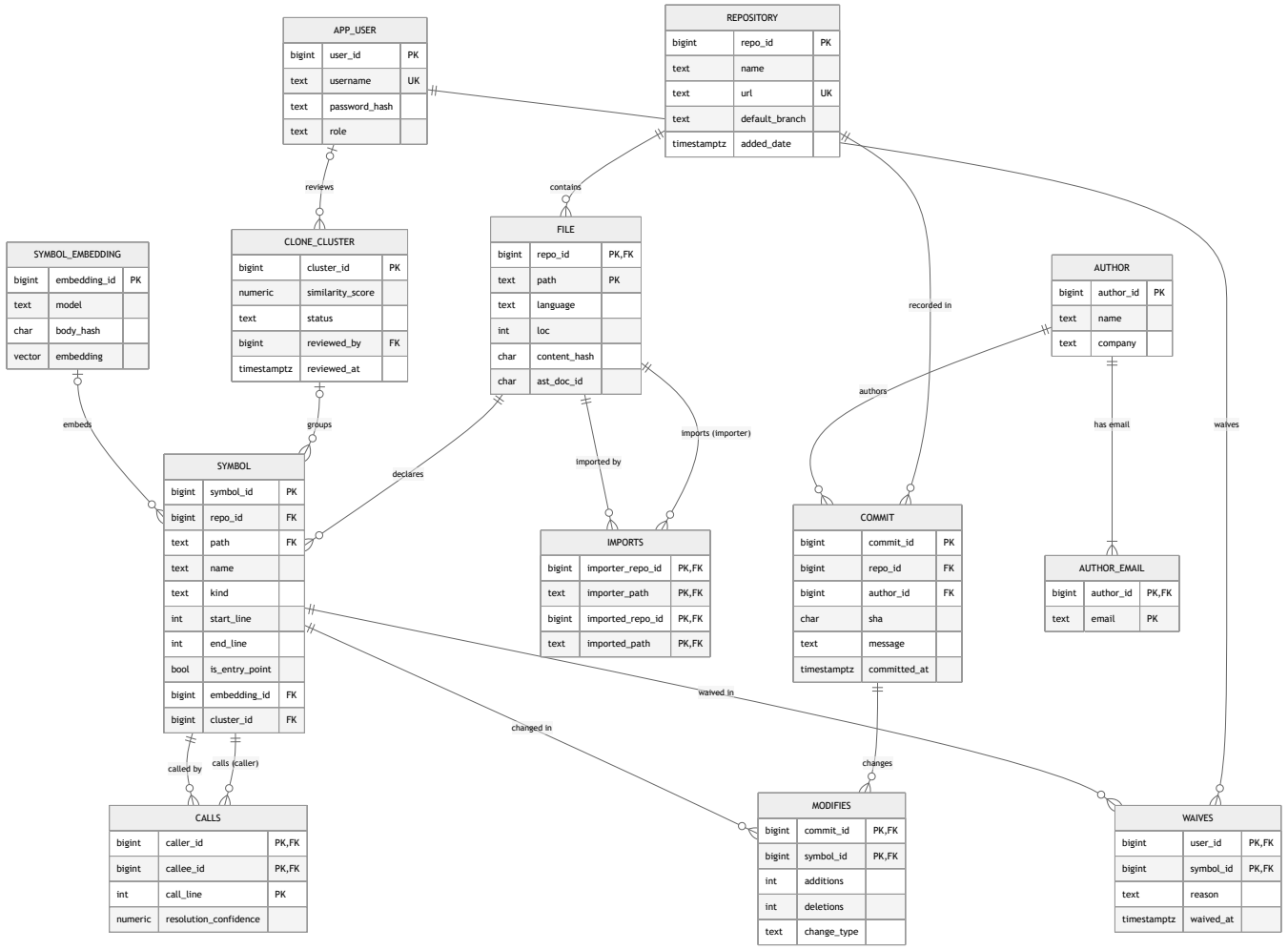
Clone detection and semantic search compare embeddings by cosine distance. pgvector adds the vector type and an HNSW index to the same PostgreSQL server, so a nearest-neighbour query joins straight to `symbol`, `file` and `clone_cluster`. Embeddings are also removed in the same transaction as their symbols. A separate vector database would have been a third server, with its IDs kept in sync with `symbol` by hand.

Need	Store	Reason
Structure, history, users	PostgreSQL 16	Fixed schema, foreign keys, transactions, recursive queries
Per-file syntax trees	MongoDB 7	Shape varies by language; read as a whole and aggregated over nodes
Symbol embeddings	pgvector	Similarity search next to the rows it joins with

5. SQL database commands

5.1 ER diagram

The full Chen-notation ER diagram and the relational schema are in ER_Relational_Schema.pdf. The diagram below shows the thirteen tables as built, with their keys.



5.2 Relational schema

`FILE` is a weak entity owned by `REPOSITORY`. `CALLS` and `IMPORTS` are recursive M:N relationships. Email is a multivalued attribute of `AUTHOR`, so it has its own table. All relations are in BCNF; the argument is in DATABASE_DESIGN.md.

Relation	Primary key	Foreign keys	Other constraints
repository	repo_id (identity)		`url` UNIQUE; name and branch not blank
file	(repo_id, path)	repo_id → repository, ON DELETE CASCADE	path relative; language in 5 values; `loc >= 0`; SHA-256 and ObjectId formats

Relation	Primary key	Foreign keys	Other constraints
symbol	symbol_id (identity)	(repo_id, path) → file; embedding_id → symbol_embedding; cluster_id → clone_cluster	kind in 6 values; `start_line >= 1`; `end_line >= start_line`; UNIQUE (repo_id, path, kind, name, start_line); span trigger
calls	(caller_id, callee_id, call_line)	caller_id, callee_id → symbol	`call_line >= 1`; confidence between 0 and 1
imports	(importer_repo_id, importer_path, imported_repo_id, imported_path)	both pairs → file	a file cannot import itself
author	author_id (identity)		name and company not blank
author_email	(author_id, email)	author_id → author	email UNIQUE, lowercase, format check
commit	commit_id (identity)	repo_id → repository; author_id → author	UNIQUE (repo_id, sha); sha is 40 hex digits
modifies	(commit_id, symbol_id)	commit_id → commit; symbol_id → symbol	additions and deletions `>= 0` and not both 0; change_type in (added, modified)
app_user	user_id (identity)		username UNIQUE and pattern; password_hash must be bcrypt; role in (admin, developer)
clone_cluster	cluster_id (identity)	reviewed_by → app_user, ON DELETE SET NULL	score between 0 and 1; status in 3 values; `reviewed_at` set exactly when closed; review trigger
waives	(user_id, symbol_id)	user_id → app_user; symbol_id → symbol	reason 5 to 500 characters
symbol_embedding	embedding_id (identity)		UNIQUE (model, body_hash); `vector(768)`; hash is 64 hex digits

5.3 DDL with constraints

From db/schema/10_structure.sql:

```
CREATE TABLE repository (
    repo_id        bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name           text        NOT NULL,
    url            text        NOT NULL,
    default_branch text        NOT NULL DEFAULT 'main',
    added_date     timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT repository_url_key UNIQUE (url),
    CONSTRAINT repository_name_not_blank CHECK (btrim(name) <> ''),
    CONSTRAINT repository_branch_not_blank CHECK (btrim(default_branch) <> '')
);

CREATE TABLE file (
    repo_id      bigint  NOT NULL,
    path         text    NOT NULL,
    language     text    NOT NULL,
```

```
    loc          integer NOT NULL DEFAULT 0,
    content_hash char(64) NOT NULL,
    ast_doc_id   char(24),
    CONSTRAINT file_pkey PRIMARY KEY (repo_id, path),
    CONSTRAINT file_repo_fkey FOREIGN KEY (repo_id)
        REFERENCES repository (repo_id) ON DELETE CASCADE,
    CONSTRAINT file_path_relative CHECK (path <> '' AND left(path, 1) <> '/' AND path !~ '\\'),
    CONSTRAINT file_language_known CHECK (language IN ('javascript', 'typescript', 'python', 'java',
'go')),
    CONSTRAINT file_loc_nonnegative CHECK (loc >= 0),
    CONSTRAINT file_content_hash_sha256 CHECK (content_hash ~ '^[0-9a-f]{64}$'),
    CONSTRAINT file_ast_doc_id_objectid CHECK (ast_doc_id ~ '^[0-9a-f]{24}$')
);

CREATE TABLE symbol (
    symbol_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    repo_id        bigint  NOT NULL,
    path           text    NOT NULL,
    name           text    NOT NULL,
    kind           text    NOT NULL,
    signature      text,
    start_line     integer NOT NULL,
    end_line       integer NOT NULL,
    is_entry_point boolean NOT NULL DEFAULT false,
    embedding_id   bigint,
    cluster_id     bigint,
    CONSTRAINT symbol_file_fkey FOREIGN KEY (repo_id, path)
        REFERENCES file (repo_id, path) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT symbol_name_not_blank CHECK (btrim(name) <> ''),
    CONSTRAINT symbol_kind_known CHECK (kind IN ('function', 'method', 'class', 'interface', 'enum',
'module')),
    CONSTRAINT symbol_start_line_positive CHECK (start_line >= 1),
    CONSTRAINT symbol_span_ordered CHECK (end_line >= start_line),
    CONSTRAINT symbol_location_key UNIQUE (repo_id, path, kind, name, start_line)
);

CREATE TABLE calls (
    caller_id             bigint       NOT NULL,
    callee_id             bigint       NOT NULL,
    call_line             integer      NOT NULL,
    resolution_confidence numeric(3,2) NOT NULL,
    CONSTRAINT calls_pkey PRIMARY KEY (caller_id, callee_id, call_line),
    CONSTRAINT calls_caller_fkey FOREIGN KEY (caller_id)
        REFERENCES symbol (symbol_id) ON DELETE CASCADE,
    CONSTRAINT calls_callee_fkey FOREIGN KEY (callee_id)
        REFERENCES symbol (symbol_id) ON DELETE CASCADE,
    CONSTRAINT calls_call_line_positive CHECK (call_line >= 1),
    CONSTRAINT calls_confidence_range CHECK (resolution_confidence BETWEEN 0 AND 1)
);

CREATE TABLE imports (
    importer_repo_id bigint NOT NULL,
    importer_path    text   NOT NULL,
    imported_repo_id bigint NOT NULL,
    imported_path    text   NOT NULL,
    CONSTRAINT imports_pkey PRIMARY KEY (importer_repo_id, importer_path, imported_repo_id, imported_path),
    CONSTRAINT imports_importer_fkey FOREIGN KEY (importer_repo_id, importer_path)
        REFERENCES file (repo_id, path) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT imports_imported_fkey FOREIGN KEY (imported_repo_id, imported_path)
        REFERENCES file (repo_id, path) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT imports_not_self CHECK (
        importer_repo_id <> imported_repo_id OR importer_path <> imported_path
    )
);
```

From db/schema/30_history.sql:

```sql
CREATE TABLE commit (
    commit_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    repo_id        bigint      NOT NULL,
    author_id      bigint      NOT NULL,
    sha            char(40)    NOT NULL,
    message        text        NOT NULL,
    committed_at   timestamptz NOT NULL,
    message_search tsvector GENERATED ALWAYS AS (to_tsvector('english', message)) STORED,
    CONSTRAINT commit_repo_sha_key UNIQUE (repo_id, sha),
    CONSTRAINT commit_repo_fkey FOREIGN KEY (repo_id)
        REFERENCES repository (repo_id) ON DELETE CASCADE,
    CONSTRAINT commit_author_fkey FOREIGN KEY (author_id)
        REFERENCES author (author_id),
    CONSTRAINT commit_sha_format CHECK (sha ~ '^[0-9a-f]{40}$')
);

CREATE TABLE modifies (
    commit_id   bigint  NOT NULL,
    symbol_id   bigint  NOT NULL,
    additions   integer NOT NULL,
    deletions   integer NOT NULL,
    change_type text    NOT NULL,
    CONSTRAINT modifies_pkey PRIMARY KEY (commit_id, symbol_id),
    CONSTRAINT modifies_commit_fkey FOREIGN KEY (commit_id)
        REFERENCES commit (commit_id) ON DELETE CASCADE,
    CONSTRAINT modifies_symbol_fkey FOREIGN KEY (symbol_id)
        REFERENCES symbol (symbol_id) ON DELETE CASCADE,
    CONSTRAINT modifies_additions_nonnegative CHECK (additions >= 0),
    CONSTRAINT modifies_deletions_nonnegative CHECK (deletions >= 0),
    CONSTRAINT modifies_changes_something CHECK (additions + deletions > 0),
    CONSTRAINT modifies_change_type_known CHECK (change_type IN ('added', 'modified'))
);

CREATE TABLE app_user (
    user_id       bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    username      text        NOT NULL,
    password_hash text        NOT NULL,
    role          text        NOT NULL,
    created_at    timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT app_user_username_key UNIQUE (username),
    CONSTRAINT app_user_username_format CHECK (username ~ '^[a-z0-9_.-]{3,32}$'),
    CONSTRAINT app_user_password_bcrypt CHECK (password_hash ~ '^\$2[aby]\$1[0-4]\$[./A-Za-z0-9]{53}$'),
    CONSTRAINT app_user_role_known CHECK (role IN ('admin', 'developer'))
);

CREATE TABLE clone_cluster (
    cluster_id       bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    similarity_score numeric(4,3) NOT NULL,
    status           text         NOT NULL DEFAULT 'open',
    detected_at      timestamptz  NOT NULL DEFAULT now(),
    reviewed_by      bigint,
    reviewed_at      timestamptz,
    CONSTRAINT clone_cluster_reviewer_fkey FOREIGN KEY (reviewed_by)
        REFERENCES app_user (user_id) ON DELETE SET NULL,
    CONSTRAINT clone_cluster_score_range CHECK (similarity_score BETWEEN 0 AND 1),
    CONSTRAINT clone_cluster_status_known CHECK (status IN ('open', 'confirmed', 'dismissed')),
    CONSTRAINT clone_cluster_reviewed_when_closed CHECK ((status = 'open') = (reviewed_at IS NULL))
);

-- symbol.cluster_id already exists as a plain column in 10_structure.sql.
ALTER TABLE symbol
    ADD CONSTRAINT symbol_cluster_fkey FOREIGN KEY (cluster_id)
        REFERENCES clone_cluster (cluster_id) ON DELETE SET NULL;
```

`author`, `author_email` and `waives` follow the same pattern; see the file for their DDL.

5.4 Indexes

Index	Table	Kind	Serves
`calls_callee_idx`	calls (callee_id)	B-tree	Each backwards step of `impact_of`
`symbol_name_idx`	symbol (name)	B-tree	Lookup by name
`symbol_entry_point_idx`	symbol (repo_id) WHERE is_entry_point	Partial B-tree	Entry points per repository
`symbol_search_idx`	symbol (search_vector)	GIN	Full-text symbol search
`imports_imported_idx`	imports (imported_repo_id, imported_path)	B-tree	Who imports this file
`commit_repo_time_idx`	commit (repo_id, committed_at DESC)	B-tree	Commit list, newest first
`commit_message_search_idx`	commit (message_search)	GIN	Commit message search
`modifies_symbol_idx`	modifies (symbol_id)	B-tree	History of one symbol
`clone_cluster_status_idx`	clone_cluster (status, detected_at DESC)	B-tree	Review queue
`symbol_embedding_hnsw_idx`	symbol_embedding (embedding vector_cosine_ops)	HNSW	Cosine nearest neighbours

5.5 DML

db/queries/dml_operations.sql runs thirteen DML steps inside one transaction and rolls back at the end. Excerpts:

```
-- 3. INSERT ... SELECT: symbols, joined to the files they live in
INSERT INTO symbol (repo_id, path, name, kind, signature, start_line, end_line, is_entry_point)
SELECT f.repo_id, f.path, v.name, v.kind, v.signature, v.start_line, v.end_line, v.entry
FROM file f
JOIN (VALUES ('src/cart.ts',    'checkout',  'function', 'function checkout(cart)',         1, 15, true),
             ('src/cart.ts',    'cartTotal', 'function', 'function cartTotal(cart)',       17, 30, false),
             ('src/tax.ts',     'applyTax',  'function', 'function applyTax(amount, rate)', 1, 10, false),
             ('jobs/report.py', 'main',      'function', 'def main()',                      1, 15, true))
     AS v (path, name, kind, signature, start_line, end_line, entry)
  ON v.path = f.path
WHERE f.repo_id = pg_temp.demo_repo()
RETURNING symbol_id, path, name, start_line, end_line, is_entry_point;

-- 6. UPSERT: adding the same URL again updates the row instead of failing, and keeps
--    its repo_id
INSERT INTO repository (name, url, default_branch)
VALUES ('dml-demo-renamed', 'test://dml-demo', 'develop')
ON CONFLICT ON CONSTRAINT repository_url_key
DO UPDATE SET name = EXCLUDED.name, default_branch = EXCLUDED.default_branch
RETURNING repo_id, name, default_branch, (repo_id = pg_temp.demo_repo()) AS same_repo_id;

-- 8. UPDATE ... FROM: halve the confidence of every call made from Python code
UPDATE calls c
SET resolution_confidence = round(c.resolution_confidence / 2, 2)
FROM symbol s
WHERE s.symbol_id = c.caller_id
  AND s.repo_id = pg_temp.demo_repo()
  AND s.path LIKE '%.py'
RETURNING s.name AS caller, c.call_line, c.resolution_confidence;
```

```
-- 11. DELETE ... USING: drop low-confidence calls made from Python files
DELETE FROM calls c
USING symbol s
WHERE s.symbol_id = c.caller_id
  AND s.repo_id = pg_temp.demo_repo()
  AND c.resolution_confidence < 0.5
RETURNING c.caller_id, c.callee_id, c.resolution_confidence;
```

Deleting one file cascades through its symbols to their calls and history rows:

```
          stage           | symbols | calls | modifies
--------------------------+---------+-------+----------
 before                   |       4 |     2 |        2
 after deleting src/tax.ts |       3 |     1 |        1
```

Six invalid writes, each in its own block, are refused by the constraints and the trigger:

```
NOTICE:  span CHECK rejected: new row for relation "symbol" violates check constraint "symbol_span_ordered"
NOTICE:  span trigger rejected: symbol applyTax spans lines 1 to 500, outside src/tax.ts, which has 25 lines
NOTICE:  language CHECK rejected: new row for relation "file" violates check constraint "file_language_known"
NOTICE:  foreign key rejected: insert or update on table "symbol" violates foreign key constraint
"symbol_file_fkey"
NOTICE:  confidence CHECK rejected: new row for relation "calls" violates check constraint
"calls_confidence_range"
NOTICE:  unique URL rejected: duplicate key value violates unique constraint "repository_url_key"
```

In normal use the ingest tools write these tables: `ingest/parse/src/store/postgres.ts` replaces a repository's structure in one transaction, and `ingest/history/src/store.ts` writes authors, commits and `modifies`.

5.6 Join, nested, correlated and recursive queries

db/queries/query_types.sql holds 22 read-only queries grouped by kind. The outputs below come from the seed repository.

Join, with `symbol` joined twice (A1):

```
SELECT caller.name AS caller, callee.name AS callee, callee.path AS callee_file,
       c.call_line, c.resolution_confidence
FROM calls c
JOIN symbol caller ON caller.symbol_id = c.caller_id
JOIN symbol callee ON callee.symbol_id = c.callee_id
JOIN repository r  ON r.repo_id = caller.repo_id
WHERE r.name = 'seed-repo' AND callee.path LIKE 'src/orders/%'
ORDER BY caller.name, c.call_line;
```

```
          caller           |      callee      |      callee_file      | call_line | resolution_confidence
---------------------------+------------------+-----------------------+-----------+-----------------------
 create                    | parseAddress     | src/orders/address.ts |        17 |                  0.90
 create                    | nextId           | src/orders/service.ts |        17 |                  0.95
 create                    | priceOrder       | src/orders/pricing.ts |        17 |                  0.90
```

```
GET                          | find            | src/orders/service.ts |         7 |                 0.85
normalizeAddress             | parseAddress    | src/orders/address.ts |        14 |                 1.00
parseAddress                 | normalizeAddress | src/orders/address.ts |         8 |                 1.00
parseAddress                 | normalizeAddress | src/orders/address.ts |         9 |                 1.00
POST                         | create          | src/orders/service.ts |        13 |                 0.85
priceOrder                   | subtotal        | src/orders/pricing.ts |        21 |                 1.00
priceOrder                   | applyDiscount   | src/orders/pricing.ts |        21 |                 1.00
src/app/api/orders/route.ts | OrderService    | src/orders/service.ts |         3 |                 0.90
```

Nested subquery with a row-value `IN` (B1):

```
SELECT s.name, s.kind, s.path
FROM symbol s
WHERE s.kind <> 'module'
  AND (s.repo_id, s.path) IN (SELECT imported_repo_id, imported_path
                              FROM imports
                              GROUP BY imported_repo_id, imported_path
                              HAVING count(*) >= 2)
ORDER BY s.path, s.start_line;
```

```
    name      |   kind   |       path
--------------+----------+------------------
 roundCurrency | function | src/lib/money.ts
 formatMoney   | function | src/lib/money.ts
```

Correlated subquery: functions and methods nothing calls that are not entry points (C1):

```
SELECT s.name, s.kind, s.path
FROM symbol s
WHERE s.kind IN ('function', 'method')
  AND NOT s.is_entry_point
  AND NOT EXISTS (SELECT 1 FROM calls c WHERE c.callee_id = s.symbol_id)
ORDER BY s.path, s.start_line;
```

```
   name    |   kind   |                 path
-----------+----------+---------------------------------------
 __init__  | method   | analytics/stats.py
 Invoice   | method   | java/src/com/acme/billing/Invoice.java
 escapeCell | function | src/legacy/format.js
```

Relational division with a double `NOT EXISTS`: authors who changed every file under `src/orders/` (C4):

```
SELECT a.name
FROM author a
WHERE NOT EXISTS (
    SELECT 1 FROM file f
    WHERE f.path LIKE 'src/orders/%'
      AND NOT EXISTS (
          SELECT 1
```

```
            FROM modifies m
            JOIN commit c ON c.commit_id = m.commit_id
            JOIN symbol s ON s.symbol_id = m.symbol_id
            WHERE c.author_id = a.author_id AND s.repo_id = f.repo_id AND s.path = f.path))
 ORDER BY a.name;
```

```
   name
------------
 Ben Okafor
```

Recursive CTE with the SQL-standard `CYCLE` clause: everything the `POST` handler ends up running (D2). `parseAddress` and `normalizeAddress` call each other, and the clause stops the recursion where a symbol repeats on the current path.

```
 WITH RECURSIVE tree (sid, depth) AS (
     SELECT s.symbol_id, 0
     FROM symbol s
     WHERE s.name = 'POST' AND s.path = 'src/app/api/orders/route.ts'
     UNION ALL
     SELECT c.callee_id, t.depth + 1
     FROM tree t
     JOIN calls c ON c.caller_id = t.sid
 ) CYCLE sid SET is_cycle USING visited
 SELECT repeat('    ', t.depth) || s.name AS call_tree, s.path, t.is_cycle
 FROM tree t
 JOIN symbol s ON s.symbol_id = t.sid
 ORDER BY t.visited;
```

```
          call_tree           |            path             | is_cycle
------------------------------+-----------------------------+----------
 POST                         | src/app/api/orders/route.ts | f
     create                   | src/orders/service.ts       | f
         logEvent             | src/lib/log.ts              | f
         formatMoney          | src/lib/money.ts            | f
             roundCurrency    | src/lib/money.ts            | f
         parseAddress         | src/orders/address.ts       | f
             normalizeAddress | src/orders/address.ts       | f
             normalizeAddress | src/orders/address.ts       | f
                 parseAddress | src/orders/address.ts       | t
                 parseAddress | src/orders/address.ts       | t
         priceOrder           | src/orders/pricing.ts       | f
             subtotal         | src/orders/pricing.ts       | f
             applyDiscount    | src/orders/pricing.ts       | f
                 roundCurrency | src/lib/money.ts           | f
         nextId               | src/orders/service.ts       | f
```

Transitive imports of the orders route, also recursive (D1):

```
         path          | depth |                                 import_chain
-----------------------+-------+-------------------------------------------------------------------------------
-
 src/lib/log.ts        |     2 | src/app/api/orders/route.ts -> src/orders/service.ts -> src/lib/log.ts
 src/lib/money.ts      |     2 | src/app/api/orders/route.ts -> src/orders/service.ts -> src/lib/money.ts
 src/orders/address.ts |     2 | src/app/api/orders/route.ts -> src/orders/service.ts -> src/orders/address.ts
 src/orders/pricing.ts |     2 | src/app/api/orders/route.ts -> src/orders/service.ts -> src/orders/pricing.ts
 src/orders/service.ts |     1 | src/app/api/orders/route.ts -> src/orders/service.ts
```

The file also has a left outer join, a full outer join, a self join, `JOIN ... USING`, scalar and derived-table subqueries, `> ALL`, `= ANY`, `INTERSECT`, `EXCEPT`, `UNION`, `GROUP BY ... HAVING` with `rank()`, and a running total over a window frame. The queries the app's features run are in STRUCTURE_QUERIES.md and HISTORY_QUERIES.md.

5.7 Views

From db/views/30_history_views.sql:

```sql
-- Symbols changed in at least two commits, with how much and by how many people.
CREATE VIEW v_symbol_hotspot AS
SELECT s.symbol_id,
       s.repo_id,
       s.path,
       s.name,
       s.kind,
       count(*)::int                      AS commits,
       count(DISTINCT c.author_id)::int   AS authors,
       sum(m.additions)::int              AS additions,
       sum(m.deletions)::int              AS deletions,
       sum(m.additions + m.deletions)::int AS churn,
       max(c.committed_at)                AS last_changed
FROM modifies m
JOIN commit c ON c.commit_id = m.commit_id
JOIN symbol s ON s.symbol_id = m.symbol_id
GROUP BY s.symbol_id, s.repo_id, s.path, s.name, s.kind
HAVING count(*) >= 2;
```

```
    name      |         path          | commits | authors | churn
--------------+-----------------------+---------+---------+-------
 OrderService | src/orders/service.ts |       3 |       1 |    20
 Totals       | analytics/stats.py    |       2 |       1 |    13
 create       | src/orders/service.ts |       3 |       1 |    10
 applyDiscount | src/orders/pricing.ts |       2 |       2 |     8
 priceOrder   | src/orders/pricing.ts |       2 |       2 |     7
```

`v_file_ownership` builds per-author churn in a CTE and ranks authors within each file with `rank() OVER (PARTITION BY ...)`. `v_author_activity` uses a `LEFT JOIN` and correlated subqueries, so authors with no commits still get a row.

5.8 Functions and the clone review procedure

`impact_of(symbol_id, max_depth)` in db/functions/20_impact_of.sql validates its arguments and then runs a chain of CTEs. `reach` is the recursive part: it walks `calls` from callee to caller.

```
    RETURN QUERY
    WITH RECURSIVE reach (sid, lvl) AS (
        SELECT p_symbol_id, 0
        UNION
        SELECT c.caller_id, r.lvl + 1
        FROM reach r
        JOIN calls c ON c.callee_id = r.sid
        WHERE r.lvl < p_max_depth
    ),
    shallowest AS (
        SELECT r.sid, min(r.lvl) AS lvl
        FROM reach r
        GROUP BY r.sid
    ),
    layered_edge AS MATERIALIZED (
        SELECT c.caller_id, c.callee_id, max(c.resolution_confidence) AS conf
        FROM calls c
        JOIN shallowest up ON up.sid = c.caller_id
        JOIN shallowest down ON down.sid = c.callee_id AND down.lvl = up.lvl - 1
        GROUP BY c.caller_id, c.callee_id
    ),
```

The `best` CTE that follows keeps, for every affected symbol, the call path with the highest weakest-link confidence. For `roundCurrency`:

```
 depth |     name      |   kind   |            path             | is_entry_point | min_confidence
-------+---------------+----------+-----------------------------+----------------+----------------
     1 | applyDiscount | function | src/orders/pricing.ts       | f              |           0.90
     1 | formatMoney   | function | src/lib/money.ts            | f              |           1.00
     2 | create        | method   | src/orders/service.ts       | f              |           0.90
     2 | priceOrder    | function | src/orders/pricing.ts       | f              |           0.90
     3 | POST          | function | src/app/api/orders/route.ts | t              |           0.85
```

`search_symbols(query, repo_id, limit)` in db/functions/21_search_symbols.sql splits camelCase and snake_case names into words, builds a prefix `tsquery`, and ranks matches on the generated `search_vector` column.

The stored procedure `review_clone_cluster` in db/functions/30_clone_review.sql closes a clone cluster and records waivers as one unit:

```
CREATE PROCEDURE review_clone_cluster(
    p_cluster_id bigint,
    p_reviewer   bigint,
    p_status     text,
    p_waive      bigint[] DEFAULT '{}',
    p_reason     text     DEFAULT NULL
)
LANGUAGE plpgsql AS $$
DECLARE
    v_status   text;
    v_outsider bigint;
BEGIN
```

```
    IF p_status IS NULL OR p_status NOT IN ('confirmed', 'dismissed') THEN
        RAISE EXCEPTION 'review outcome must be confirmed or dismissed, not %', p_status
            USING ERRCODE = 'invalid_parameter_value';
    END IF;

    SELECT status INTO v_status FROM clone_cluster WHERE cluster_id = p_cluster_id FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'clone cluster % does not exist', p_cluster_id USING ERRCODE = 'no_data_found';
    END IF;
    IF v_status <> 'open' THEN
        RAISE EXCEPTION 'clone cluster % was already reviewed', p_cluster_id USING ERRCODE =
'serialization_failure';
    END IF;
```

5.9 Triggers

Trigger	Table and timing	Rule
`symbol_within_file`	`symbol`, BEFORE INSERT OR UPDATE of the span columns	A symbol's lines must fall inside its file's `loc`
`file_keeps_symbols`	`file`, BEFORE UPDATE OF loc, WHEN the file shrinks	A file cannot shrink below the end of its last symbol
`clone_cluster_review_guard`	`clone_cluster`, BEFORE INSERT OR UPDATE OF status	Only an `app_user` with role admin can close a cluster; closing stamps `reviewed_at`, reopening clears the reviewer

```
CREATE FUNCTION clone_cluster_review_guard() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'UPDATE' AND NEW.status = OLD.status THEN
        RETURN NEW;
    END IF;

    -- Unknown statuses are left to the CHECK constraint.
    IF NEW.status NOT IN ('confirmed', 'dismissed') THEN
        IF TG_OP = 'UPDATE' AND NEW.status = 'open' THEN
            NEW.reviewed_by := NULL;
            NEW.reviewed_at := NULL;
        END IF;
        RETURN NEW;
    END IF;

    IF NEW.reviewed_by IS NULL THEN
        RAISE EXCEPTION 'closing clone cluster % needs a reviewer', NEW.cluster_id
            USING ERRCODE = 'insufficient_privilege';
    END IF;

    IF NOT EXISTS (SELECT 1 FROM app_user WHERE user_id = NEW.reviewed_by AND role = 'admin') THEN
        RAISE EXCEPTION 'user % is not an admin and cannot review clone clusters', NEW.reviewed_by
            USING ERRCODE = 'insufficient_privilege';
    END IF;

    NEW.reviewed_at := now();
    RETURN NEW;
END $$;
```

5.10 CTEs, transactions and locking

CTEs appear throughout: the recursive `reach` and the `best` path selection inside `impact_of`, `per_author` inside `v_file_ownership`, the two-direction call graph in structure_queries.sql, and data-modifying CTEs that insert an author, its emails, a commit and its `modifies` rows in one statement (DML step 5).

Transactions:

- Parsing a repository writes its files, symbols, calls and imports in one transaction. If anything fails, it rolls back and deletes the Mongo documents created during the run.
- `review_clone_cluster` runs inside the caller's transaction. `SELECT ... FOR UPDATE` locks the cluster row, so a second admin reviewing at the same time waits and then gets SQLSTATE 40001 instead of overwriting the first review.
- The semantic search page runs its vector query in a `BEGIN READ ONLY` transaction with `SET LOCAL hnsw.ef_search = 100`.

5.11 Role-based access control in the database

db/security/40_roles.sql creates database roles that mirror the two app roles:

```
-- An admin inherits every developer privilege.
GRANT cie_developer TO cie_admin;

GRANT USAGE ON SCHEMA public TO cie_developer;

-- Developers: read-only access to the code structure, history and similarity data.
GRANT SELECT ON repository, file, symbol, calls, imports,
                author, author_email, commit, modifies,
                clone_cluster, waives, symbol_embedding
    TO cie_developer;
GRANT SELECT ON v_symbol_hotspot, v_file_ownership, v_author_activity TO cie_developer;

-- Column-level grant: developers can see who the users are and what role they have,
-- never the password hash.
GRANT SELECT (user_id, username, role, created_at) ON app_user TO cie_developer;

-- Admins: add, re-ingest and remove repositories. Re-ingesting deletes and recreates
-- the repository's files, symbols, calls and imports.
GRANT INSERT, UPDATE, DELETE ON repository, file, symbol, calls, imports TO cie_admin;

-- Admins: review clone clusters. Only the two columns a review sets are writable;
-- reviewed_at is stamped by the clone_cluster_review_guard trigger.
GRANT UPDATE (status, reviewed_by) ON clone_cluster TO cie_admin;
GRANT INSERT, UPDATE, DELETE ON waives TO cie_admin;

-- Admins: manage users, including their password hashes.
GRANT SELECT, INSERT, UPDATE, DELETE ON app_user TO cie_admin;

-- Functions are executable by PUBLIC by default. The read functions stay that way.
-- The review procedure is limited to admins, so a developer cannot even call it.
REVOKE EXECUTE ON PROCEDURE review_clone_cluster(bigint, bigint, text, bigint[], text) FROM PUBLIC;
GRANT EXECUTE ON PROCEDURE review_clone_cluster(bigint, bigint, text, bigint[], text) TO cie_admin;
```

```
cie_demo=> SET ROLE cie_developer;
cie_demo=> SELECT password_hash FROM app_user;
ERROR:  permission denied for table app_user
cie_demo=> INSERT INTO repository (name, url) VALUES ('x', 'test://x');
```

```
ERROR:  permission denied for table repository
cie_demo=> CALL review_clone_cluster(1, 1, 'confirmed');
ERROR:  permission denied for procedure review_clone_cluster
cie_demo=> SET ROLE cie_admin;
cie_demo=> INSERT INTO repository (name, url) VALUES ('x', 'test://x') RETURNING name, url;
 name |   url
------+----------
 x    | test://x
```

db/tests/40_roles_test.sql runs 34 statements under these roles and checks that each one succeeds or fails with SQLSTATE 42501, as expected.

6. NoSQL and vector database commands

6.1 MongoDB: creating the `ast` collection

Each parsed file is one document in `ast`. The tree is stored flat as an array of nodes: `i` is the node's index, `p` its parent's index, `t` its type, `d` its depth, and `s` and `e` its start and end positions. A nested tree would hit BSON's limit of 100 nesting levels on deep files; the flat array never does, and pipelines can `$unwind` the nodes directly. Anonymous nodes such as punctuation are dropped, and the array is capped at 50,000 entries. db/mongo/init/10_ast.js creates the collection with a `$jsonSchema` validator and its indexes:

```
const target = db.getSiblingDB(dbName);
const validation = {
  validator: { $jsonSchema: schema },
  validationLevel: 'moderate',
  validationAction: 'error',
};

if (target.getCollectionNames().includes('ast')) {
  target.runCommand({ collMod: 'ast', ...validation });
} else {
  target.createCollection('ast', validation);
}
```

Part of the validator, from db/mongo/ast.schema.json:

```
{
  "bsonType": "object",
  "required": [
    "repo_id",
    "path",
    "language",
    "content_hash",
    "parser",
    "parsed_at",
    "loc",
    "node_count",
    "max_depth",
    "truncated",
    "nodes"
  ],
  "additionalProperties": false,
  "properties": {
    "_id": { "bsonType": "objectId" },
    "repo_id": { "bsonType": ["int", "long"], "minimum": 1 },
    "path": { "bsonType": "string", "minLength": 1, "pattern": "^[^/\\\\][^\\\\]*$" },
    "language": { "enum": ["javascript", "typescript", "python", "java", "go"] },
    "content_hash": { "bsonType": "string", "pattern": "^[0-9a-f]{64}$" },
```

Indexes, from db/mongo/ast.indexes.json:

```
[
  { "name": "repo_path_unique", "key": { "repo_id": 1, "path": 1 }, "unique": true },
  { "name": "repo_depth", "key": { "repo_id": 1, "max_depth": -1, "path": 1 } },
  { "name": "repo_language", "key": { "repo_id": 1, "language": 1 } },
  { "name": "language", "key": { "language": 1 } },
```

```
    { "name": "content_hash", "key": { "content_hash": 1 } }
  ]
```

6.2 MongoDB aggregation pipelines

db/mongo/pipelines.js defines four pipelines: `nodeTypeCountsByLanguage`, `deepestNestingPerFile`, `astSummaryByRepo` and `fileSizeDistribution`. The repository page calls them through lib/structure/ast.ts. The first counts node types per language with `$unwind`, three `$group` stages and `$topN`:

```
function nodeTypeCountsByLanguage({ repoId, limit } = {}) {
  const top = positiveInt(limit, 10, 'limit');
  return [
    ...matchRepo(repoId),
    { $project: { _id: 1, language: 1, types: '$nodes.t' } },
    { $unwind: '$types' },
    {
      $group: {
        _id: { language: '$language', type: '$types', file: '$_id' },
        count: { $sum: 1 },
      },
    },
    {
      $group: {
        _id: { language: '$_id.language', type: '$_id.type' },
        count: { $sum: '$count' },
        files: { $sum: 1 },
      },
    },
    {
      $group: {
        _id: '$_id.language',
        total_nodes: { $sum: '$count' },
        distinct_types: { $sum: 1 },
        top_types: {
          $topN: {
            n: top,
            sortBy: { count: -1, '_id.type': 1 },
            output: { type: '$_id.type', count: '$count', files: '$files' },
          },
        },
      },
    },
```

Output for the seed repository with `limit` 3 (first language shown):

```
{
  total_nodes: 685,
  distinct_types: 56,
  language: 'typescript',
  top_types: [
    { type: 'identifier', count: 124, files: 6, share: 0.181 },
    { type: 'property_identifier', count: 60, files: 6, share: 0.0876 },
    { type: 'type_annotation', count: 41, files: 6, share: 0.0599 }
  ]
}
```

`deepestNestingPerFile` finds the deepest node in each file and rebuilds its chain of ancestors with `$reduce` over the parent pointers:

```
{
  path: 'src/lib/log.ts',
  max_depth: 14,
  deepest_node: { type: 'identifier', line: 2, column: 56 },
  ancestry: [
    'program', 'export_statement', 'function_declaration', 'statement_block',
    'expression_statement', 'call_expression', 'arguments', 'call_expression',
    'arguments', 'object', 'pair', 'call_expression', 'member_expression', 'new_expression'
  ]
}
```

6.3 MongoDB read, update and delete

Documents are created by the ingester (ingest/parse/src/store/mongo.ts), which upserts by `(repo_id, path)`. db/mongo/rud.js reads the real documents and runs updates and deletes on two scratch copies, which it removes at the end.

```
heading('R2. find with a range filter, sort and limit: most deeply nested files');
ast
  .find({ repo_id: source.repo_id, max_depth: { $gte: 10 } }, { _id: 0, path: 1, language: 1, max_depth: 1,
node_count: 1 })
  .sort({ max_depth: -1, path: 1 })
  .limit(5)
  .forEach((doc) => printjson(doc));
```

```
index used: "repo_depth", documents examined: 14
```

```
heading('U2. updateOne with an aggregation pipeline: cut the tree at depth 8 and recompute');
const show = { _id: 0, node_count: 1, max_depth: 1, truncated: 1 };
printjson(ast.findOne({ _id: copy._id }, show));
printjson(
  ast.updateOne({ _id: copy._id }, [
    { $set: { nodes: { $filter: { input: '$nodes', cond: { $lte: ['$$this.d', 8] } } } } },
    { $set: { node_count: { $toInt: { $size: '$nodes' } }, max_depth: { $toInt: { $max: '$nodes.d' } },
truncated: true } },
  ]),
);
printjson(ast.findOne({ _id: copy._id }, show));
```

```
{ node_count: 135, max_depth: 11, truncated: false }
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
{ node_count: 117, max_depth: 8, truncated: true }
```

Step	Operation	Result
R1	`findOne` with a projection that drops `nodes`	File metadata only
R3	`distinct('language')` and `countDocuments`	go 2, java 2, javascript 2, python 4, typescript 6

Step	Operation	Result
R4	`$elemMatch` on `nodes`	4 files declare a class
R5	`$or` with `$in` and `$gt`	Go or Java files, or files over 25 lines
U1	`updateOne` with `$set` and `$inc`	`loc` 22 to 25
U3	`findOneAndUpdate`, `returnDocument: 'after'`	Changed `parser` sub-document
U4	`updateOne` with `upsert: true`	Matches the existing copy
U5	`updateMany`	2 matched
U6	Invalid writes	Language `cobol` and `loc` -1 fail validation (code 121); a duplicate `(repo_id, path)` fails the unique index (code 11000)
D1	`deleteOne` by `_id`	1 deleted
D2	`deleteMany` by filter	1 deleted; 0 scratch documents left, 16 real documents unchanged

6.4 pgvector: creation

From db/schema/31_vectors.sql:

```
CREATE EXTENSION IF NOT EXISTS vector;

-- One embedding per distinct symbol body and model. Symbols with identical bodies
-- (or a symbol that survives a re-parse unchanged) share a row through body_hash.
CREATE TABLE symbol_embedding (
    embedding_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    model        text        NOT NULL,
    body_hash    char(64)    NOT NULL,
    embedding    vector(768) NOT NULL,
    created_at   timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT symbol_embedding_model_body_key UNIQUE (model, body_hash),
    CONSTRAINT symbol_embedding_body_hash_hex CHECK (body_hash ~ '^[0-9a-f]{64}$'),
    CONSTRAINT symbol_embedding_model_not_blank CHECK (btrim(model) <> '')
);

-- Approximate nearest-neighbour search by cosine distance (the <=> operator).
CREATE INDEX symbol_embedding_hnsw_idx ON symbol_embedding
    USING hnsw (embedding vector_cosine_ops);

-- symbol.embedding_id already exists as a plain column in 10_structure.sql.
ALTER TABLE symbol
    ADD CONSTRAINT symbol_embedding_fkey FOREIGN KEY (embedding_id)
        REFERENCES symbol_embedding (embedding_id) ON DELETE SET NULL;
```

6.5 pgvector: reads, writes and features

db/queries/vector_operations.sql, nearest neighbours by cosine distance (V2):

```
SELECT s.name, s.path,
       round((e.embedding <=> q.embedding)::numeric, 4) AS cosine_distance,
       round((1 - (e.embedding <=> q.embedding))::numeric, 4) AS similarity
FROM symbol_embedding e
JOIN symbol s ON s.embedding_id = e.embedding_id
```

```
  CROSS JOIN (SELECT e2.embedding, e2.embedding_id
              FROM symbol t JOIN symbol_embedding e2 ON e2.embedding_id = t.embedding_id
              WHERE t.name = 'priceOrder') AS q
  WHERE e.embedding_id <> q.embedding_id
  ORDER BY e.embedding <=> q.embedding
  LIMIT 5;
```

```
     name      |         path          | cosine_distance | similarity
---------------+-----------------------+-----------------+------------
 applyDiscount | src/orders/pricing.ts |          0.2837 |     0.7163
 subtotal      | src/orders/pricing.ts |          0.3665 |     0.6335
 create        | src/orders/service.ts |          0.3700 |     0.6300
 line_total    | analytics/stats.py    |          0.4255 |     0.5745
 roundCurrency | src/lib/money.ts      |          0.5527 |     0.4473
```

The plan for a k-NN search uses the HNSW index (V4):

```
 Limit
   InitPlan 1 (returns $0)
     ->  Limit
           ->  Index Scan using symbol_embedding_pkey on symbol_embedding symbol_embedding_1
   ->  Index Scan using symbol_embedding_hnsw_idx on symbol_embedding
         Order By: (embedding <=> $0)
```

The most similar pairs are the candidates clone detection groups into clusters (V5):

```
     first     |      second      | similarity
---------------+------------------+------------
 parseAddress  | normalizeAddress |     0.7548
 main          | run              |     0.7538
 roundCurrency | formatMoney      |     0.7406
 applyDiscount | priceOrder       |     0.7163
 add           | line_total       |     0.7081
```

Create, update and delete run in a transaction that rolls back:

```
  -- V8. Update: replace a stored vector in place, here by scaling it. Cosine distance
  --     ignores length, so the neighbours do not change; Euclidean distance does.
  UPDATE symbol_embedding
  SET embedding = (SELECT array_agg(x * 2 ORDER BY ord) FROM unnest(embedding::real[]) WITH ORDINALITY AS u
  (x, ord))::vector
  WHERE body_hash = encode(sha256('priceOrder v2'), 'hex')
  RETURNING embedding_id, round(vector_norm(embedding)::numeric, 3) AS new_norm;

  -- V9. Delete: removing a vector unlinks its symbols through ON DELETE SET NULL; the
  --     symbol itself stays.
  DELETE FROM symbol_embedding
  WHERE body_hash = encode(sha256('priceOrder v2'), 'hex')
  RETURNING embedding_id;
```

Step	Operation	Result
V1	Count, `vector_dims`, `vector_norm`	34 vectors, 768 dimensions, unit length
V3	`<=>`, `<->` and `<#>` side by side	Same order, because the vectors are normalised
V6	`avg(embedding)` per file, closest function to the centroid	For example `priceOrder` in `src/orders/pricing.ts`
V7	Insert a vector and point `priceOrder` at it	New `embedding_id`, 768 dimensions
V8	Scale it by 2	Norm 2.000
V9	Delete it	`priceOrder.embedding_id` becomes NULL
V10	Invalid vectors	`expected 768 dimensions, not 3`; malformed hash fails the CHECK

The app's semantic search (lib/search/semantic.ts) embeds the user's description with the same model and runs the same `ORDER BY e.embedding <=> $1::vector` query, optionally filtered by repository.

7. Next.js project tree structure

Top level of the repository:

```
.
├── app/                Next.js App Router: pages, server actions, /api route handlers
├── components/         React components, by feature (app, structure, history, clones)
├── lib/                Server-side code: database clients, queries, auth
├── middleware.ts       Session and role check on every request
├── db/
│   ├── schema/         DDL: tables, constraints, indexes, pgvector table
│   ├── functions/      impact_of, search_symbols, triggers, review procedure
│   ├── views/          hotspot, ownership and author activity views
│   ├── security/       database roles and grants
│   ├── seed/           seed repository rows
│   ├── queries/        documented queries, DML, query kinds, vector operations
│   ├── mongo/          ast validator, indexes, pipelines, read/update/delete script
│   ├── init/           applies every SQL folder on the first container start
│   └── tests/          SQL tests, run in a scratch database
├── ingest/             Command line tools: parse, history, embed
├── scripts/            create-user
├── tests/              App unit tests and API smoke tests
├── docs/               Design documents, query docs, test results, screenshots
└── docker-compose.yml  PostgreSQL with pgvector, and MongoDB
```

The Next.js part:

```
app/
├── admin/users/            Admin-only user management (page and server action)
├── api/                    JSON route handlers, one per screen
│   ├── clones/             GET queue; GET and POST /clones/[clusterId] (review)
│   ├── commits/            GET list; GET /commits/[commitId]
│   ├── history/            authors, hotspots, summary
│   ├── repositories/       GET list, POST add; [repoId], ast-stats, files/[...path]
│   ├── search/             semantic search (pgvector)
│   ├── session/            JSON sign in and sign out
│   └── symbols/            search; [symbolId], graph, impact
├── clones/                 Clone review queue and cluster page with review action
├── commits/                Commit history and commit detail
├── dashboard/              Totals, hotspots, churn per author
├── impact/                 Impact analysis
├── login/                  Sign-in page and action
├── logout/                 Clears the session cookie
├── repositories/           Repository list, add form, repository page, file page
├── search/                 Search by meaning
├── symbols/[symbolId]/     Symbol detail and call graph
├── globals.css
```

```
├── layout.tsx
└── page.tsx                  Redirects to the dashboard
components/
├── app/                      LoginForm, UserMenu, AddUserForm
├── clones/                   Clone review styles
├── history/                  AuthorChurnChart, HotspotTable, RepoFilter, StatTiles
└── structure/                Shell, FileTree, CallTable, CallGraphView, graphLayout,
                              AstStatsPanel, AddRepositoryForm, ImpactView,
                              WorkflowGraph, workflowLayout, badges
lib/
├── auth/                     access rules, bcrypt, signed sessions, login throttle, users
├── clones/                   clone queue and review queries
├── history/                  history queries and filters
├── search/                   query embedding and vector search
├── structure/                repositories, files, symbols, graph, impact, AST stats, ingest
├── env.ts                    required environment variables
├── mongo.ts                  cached MongoDB client
└── pg.ts                     shared PostgreSQL pool
```

API routes:

Method and path	Purpose
`POST /api/session`, `DELETE /api/session`	Sign in and set the session cookie; sign out
`GET /api/repositories`, `POST /api/repositories`	List repositories; add one (admin)
`GET /api/repositories/[repoId]`	Repository with its file tree
`GET /api/repositories/[repoId]/files/[...path]`	One file and its symbols
`GET /api/repositories/[repoId]/ast-stats`	MongoDB aggregation results
`GET /api/symbols/search`	Full-text symbol search
`GET /api/symbols/[symbolId]`	Symbol with callers and callees
`GET /api/symbols/[symbolId]/graph`	Call graph around a symbol
`GET /api/symbols/[symbolId]/impact`	`impact_of` result and summary
`GET /api/history/summary`, `/hotspots`, `/authors`	Dashboard data from the views
`GET /api/commits`, `GET /api/commits/[commitId]`	Commit list and commit detail
`GET /api/search`	Semantic search by cosine similarity
`GET /api/clones`, `GET /api/clones/[clusterId]`, `POST /api/clones/[clusterId]`	Clone queue, cluster, review (admin)
`POST /logout`	Sign out from the page menu

8. Sample screenshots

8.1 Application code

db/functions/20_impact_of.sql

```sql
CREATE OR REPLACE FUNCTION impact_of(p_symbol_id bigint, p_max_depth integer DEFAULT 5)
RETURNS TABLE (
    symbol_id      bigint,
    repo_id        bigint,
    path           text,
    name           text,
    kind           text,
    depth          integer,
    is_entry_point boolean,
    min_confidence numeric,
    call_path      bigint[]
)
LANGUAGE plpgsql
STABLE
AS $$
#variable_conflict use_column
BEGIN
    IF p_max_depth IS NULL OR p_max_depth < 1 OR p_max_depth > 50 THEN
        RAISE EXCEPTION 'max_depth must be between 1 and 50, got %', coalesce(p_max_depth::text, 'NULL')
            USING ERRCODE = 'invalid_parameter_value';
    END IF;

    IF NOT EXISTS (SELECT 1 FROM symbol s WHERE s.symbol_id = p_symbol_id) THEN
        RAISE EXCEPTION 'symbol % does not exist', p_symbol_id
            USING ERRCODE = 'no_data_found';
    END IF;

    RETURN QUERY
    WITH RECURSIVE reach (sid, lvl) AS (
        SELECT p_symbol_id, 0
        UNION
        SELECT c.caller_id, r.lvl + 1
        FROM reach r
        JOIN calls c ON c.callee_id = r.sid
        WHERE r.lvl < p_max_depth
    ),
    shallowest AS (
        SELECT r.sid, min(r.lvl) AS lvl
        FROM reach r
        GROUP BY r.sid
    ),
    layered_edge AS MATERIALIZED (
        SELECT c.caller_id, c.callee_id, max(c.resolution_confidence) AS conf
        FROM calls c
        JOIN shallowest up ON up.sid = c.caller_id
```

`impact_of`: argument checks, then the recursive CTE that walks `calls` backwards.

db/security/40_roles.sql

```sql
-- An admin inherits every developer privilege.
GRANT cie_developer TO cie_admin;

GRANT USAGE ON SCHEMA public TO cie_developer;

-- Developers: read-only access to the code structure, history and similarity data.
GRANT SELECT ON repository, file, symbol, calls, imports,
                author, author_email, commit, modifies,
                clone_cluster, waives, symbol_embedding
    TO cie_developer;
GRANT SELECT ON v_symbol_hotspot, v_file_ownership, v_author_activity TO cie_developer;

-- Column-level grant: developers can see who the users are and what role they have,
-- never the password hash.
GRANT SELECT (user_id, username, role, created_at) ON app_user TO cie_developer;

-- Admins: add, re-ingest and remove repositories. Re-ingesting deletes and recreates
-- the repository's files, symbols, calls and imports.
GRANT INSERT, UPDATE, DELETE ON repository, file, symbol, calls, imports TO cie_admin;

-- Admins: review clone clusters. Only the two columns a review sets are writable;
-- reviewed_at is stamped by the clone_cluster_review_guard trigger.
GRANT UPDATE (status, reviewed_by) ON clone_cluster TO cie_admin;
GRANT INSERT, UPDATE, DELETE ON waives TO cie_admin;

-- Admins: manage users, including their password hashes.
GRANT SELECT, INSERT, UPDATE, DELETE ON app_user TO cie_admin;

-- Functions are executable by PUBLIC by default. The read functions stay that way.
-- The review procedure is limited to admins, so a developer cannot even call it.
REVOKE EXECUTE ON PROCEDURE review_clone_cluster(bigint, bigint, text, bigint[], text) FROM PUBLIC;
GRANT EXECUTE ON PROCEDURE review_clone_cluster(bigint, bigint, text, bigint[], text) TO cie_admin;

```

Grants for `cie_developer` and `cie_admin`, with a column-level grant on `app_user`.

db/functions/30_clone_review.sql

```sql
-- Only an admin can confirm or dismiss a clone cluster. Closing a cluster stamps
-- reviewed_at; reopening it clears the reviewer.
CREATE FUNCTION clone_cluster_review_guard() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'UPDATE' AND NEW.status = OLD.status THEN
        RETURN NEW;
    END IF;

    -- Unknown statuses are left to the CHECK constraint.
    IF NEW.status NOT IN ('confirmed', 'dismissed') THEN
        IF TG_OP = 'UPDATE' AND NEW.status = 'open' THEN
            NEW.reviewed_by := NULL;
            NEW.reviewed_at := NULL;
        END IF;
        RETURN NEW;
    END IF;

    IF NEW.reviewed_by IS NULL THEN
        RAISE EXCEPTION 'closing clone cluster % needs a reviewer', NEW.cluster_id
            USING ERRCODE = 'insufficient_privilege';
    END IF;

    IF NOT EXISTS (SELECT 1 FROM app_user WHERE user_id = NEW.reviewed_by AND role = 'admin') THEN
        RAISE EXCEPTION 'user % is not an admin and cannot review clone clusters', NEW.reviewed_by
            USING ERRCODE = 'insufficient_privilege';
    END IF;

    NEW.reviewed_at := now();
    RETURN NEW;
END $$;

CREATE TRIGGER clone_cluster_review_guard
    BEFORE INSERT OR UPDATE OF status ON clone_cluster
    FOR EACH ROW EXECUTE FUNCTION clone_cluster_review_guard();

```

The trigger that lets only admins close a clone cluster and stamps `reviewed_at`.

lib/auth/access.ts

```ts
// Who may do what. Developers can read everything; only admins can add repositories
// (the JSON route and the page's server action both POST), review clone clusters, or
// manage users.
export function accessFor(method: string, pathname: string, user: SessionUser | null): Access {
  if (PUBLIC.some((pattern) => pattern.test(pathname))) return "allow";
  if (!user) return under(pathname, "/api") ? 401 : "login";
  if (user.role === "admin") return "allow";

  if (under(pathname, "/admin")) return 403;
  if (!isRead(method)) {
    for (const prefix of ["/repositories", "/api/repositories", "/clones", "/api/clones"]) {
      if (under(pathname, prefix)) return 403;
    }
  }
  return "allow";
}

```

middleware.ts

```ts
import { NextResponse, type NextRequest } from "next/server";
import { accessFor } from "@/lib/auth/access";
import { readSessionToken, SESSION_COOKIE } from "@/lib/auth/session";

export async function middleware(request: NextRequest) {
  const { pathname, search } = request.nextUrl;
  const user = await readSessionToken(request.cookies.get(SESSION_COOKIE)?.value);
  const access = accessFor(request.method, pathname, user);

  if (access === "allow") return NextResponse.next();
  if (access === "login") {
    const login = new URL("/login", request.url);
    login.searchParams.set("next", `${pathname}${search}`);
    return NextResponse.redirect(login);
  }
  const message = access === 401 ? "Sign in first." : "Only admins can do that.";
  return NextResponse.json({ error: message }, { status: access });
}

```

Role rules in `lib/auth/access.ts` and the middleware that applies them to every request.

app/api/symbols/[symbolId]/impact/route.ts

```ts
import type { NextRequest } from "next/server";
import { handle } from "@/lib/structure/http";
import { IMPACT_MAX_DEPTH, impactOf, summarizeImpact } from "@/lib/structure/impact";
import { parseBoundedInt, parseConfidence, parseId } from "@/lib/structure/params";

export function GET(request: NextRequest, { params }: { params: Promise<{ symbolId: string }> }) {
  return handle(async () => {
    const symbolId = parseId((await params).symbolId, "symbolId");
    const query = request.nextUrl.searchParams;
    const depth = parseBoundedInt(query.get("depth"), "depth", { min: 1, max: IMPACT_MAX_DEPTH, fallback: 5 });
    const minConfidence = parseConfidence(query.get("min"));
    const affected = await impactOf(symbolId, depth, minConfidence);
    return { symbolId, depth, minConfidence, summary: summarizeImpact(affected), affected };
  });
}
```

lib/structure/impact.ts

```ts
export async function impactOf(symbolId: number, depth: number, minConfidence = 0): Promise<ImpactRow[]> {
  const { rows } = await pool.query<{
    symbol_id: string;
    repo_id: string;
    path: string;
    name: string;
    kind: SymbolKind;
    depth: number;
    is_entry_point: boolean;
    min_confidence: string;
    call_path: string[];
  }>(
    `SELECT symbol_id, repo_id, path, name, kind, depth, is_entry_point, min_confidence, call_path
     FROM impact_of($1, $2)
     WHERE min_confidence >= $3`,
    [symbolId, depth, minConfidence],
  );
```

The impact route handler and the query that calls `impact_of`.

lib/search/semantic.ts

```ts
export async function searchByMeaning(
  query: string,
  { repoId, limit }: { repoId: number | null; limit: number },
): Promise<SearchResult[]> {
  const vector = `[${(await embedQuery(query)).join(",")}]`;
  const client = await pool.connect();
  try {
    await client.query("BEGIN READ ONLY");
    // A wider HNSW candidate list than the default 40, and keep scanning when the
    // repository filter drops candidates.
    await client.query("SET LOCAL hnsw.ef_search = 100");
    await client.query("SET LOCAL hnsw.iterative_scan = relaxed_order");
    const result = await client.query<{
      symbol_id: string;
      repo_id: string;
      repo_name: string;
      path: string;
      name: string;
      kind: SymbolKind;
      signature: string | null;
      start_line: number;
      end_line: number;
      similarity: number;
    }>(
      `SELECT s.symbol_id, s.repo_id, r.name AS repo_name, s.path, s.name, s.kind, s.signature,
              s.start_line, s.end_line, (1 - (e.embedding <=> $1::vector))::float8 AS similarity
       FROM symbol_embedding e
       JOIN symbol s ON s.embedding_id = e.embedding_id
       JOIN repository r ON r.repo_id = s.repo_id
       WHERE e.model = $2 AND ($3::bigint IS NULL OR s.repo_id = $3)
       ORDER BY e.embedding <=> $1::vector, s.symbol_id
       LIMIT $4`,
      [vector, EMBEDDING_MODEL, repoId, limit],
    );
    await client.query("COMMIT");
```

Semantic search: a read-only transaction ordering by cosine distance with `<=>`.

db/mongo/pipelines.js

```
function nodeTypeCountsByLanguage({ repoId, limit } = {}) {
  const top = positiveInt(limit, 10, 'limit');
  return [
    ...matchRepo(repoId),
    { $project: { _id: 1, language: 1, types: '$nodes.t' } },
    { $unwind: '$types' },
    {
      $group: {
        _id: { language: '$language', type: '$types', file: '$_id' },
        count: { $sum: 1 },
      },
    },
    {
      $group: {
        _id: { language: '$_id.language', type: '$_id.type' },
        count: { $sum: '$count' },
        files: { $sum: 1 },
      },
    },
    {
      $group: {
        _id: '$_id.language',
        total_nodes: { $sum: '$count' },
        distinct_types: { $sum: 1 },
        top_types: {
          $topN: {
            n: top,
            sortBy: { count: -1, '_id.type': 1 },
            output: { type: '$_id.type', count: '$count', files: '$files' },
          },
        },
      },
    },
    {
```

The node-type aggregation pipeline over the `ast` collection.

db/mongo/rud.js

```
heading('U2. updateOne with an aggregation pipeline: cut the tree at depth 8 and recompute');
const show = { _id: 0, node_count: 1, max_depth: 1, truncated: 1 };
printjson(ast.findOne({ _id: copy._id }, show));
printjson(
  ast.updateOne({ _id: copy._id }, [
    { $set: { nodes: { $filter: { input: '$nodes', cond: { $lte: ['$$this.d', 8] } } } } },
    { $set: { node_count: { $toInt: { $size: '$nodes' } }, max_depth: { $toInt: { $max: '$nodes.d' } }, truncated: true } },
  ]),
);
printjson(ast.findOne({ _id: copy._id }, show));
```

db/mongo/rud.js

```
heading('U6. The validator and the unique index refuse bad writes');
function refused(label, write) {
  try {
    write();
    print(`${label}: accepted`);
  } catch (error) {
    print(`${label} rejected: code ${error.code}, ${error.message.split('\n')[0].slice(0, 70)}`);
  }
}
refused('language cobol', () => ast.updateOne({ _id: copy._id }, { $set: { language: 'cobol' } }));
refused('negative loc', () => ast.updateOne({ _id: copy._id }, { $set: { loc: NumberInt(-1) } }));
```

A pipeline update on an AST document, and writes the validator refuses.

8.2 Database demo

SQL test suite (scratch database)

```
$ docker compose exec -T postgres sh /cie-db/tests/run.sh
pass  20_impact_of_test.sql
pass  21_symbol_search_test.sql
pass  22_symbol_span_trigger_test.sql
pass  30_history_schema_test.sql
pass  31_vectors_test.sql
pass  32_history_views_test.sql
pass  33_clone_review_test.sql
pass  40_roles_test.sql
```

All eight SQL test files pass in a scratch database.

psql: recursive CTE with CYCLE detection

```
cie=# -- D2. Downstream call tree from POST (recursive CTE with CYCLE)
          call_tree          |           path            | is_cycle
-----------------------------+---------------------------+----------
 POST                        | src/app/api/orders/route.ts | f
     create                  | src/orders/service.ts       | f
         logEvent            | src/lib/log.ts              | f
         formatMoney         | src/lib/money.ts            | f
             roundCurrency   | src/lib/money.ts            | f
         parseAddress        | src/orders/address.ts       | f
             normalizeAddress | src/orders/address.ts      | f
             normalizeAddress | src/orders/address.ts      | f
                 parseAddress | src/orders/address.ts      | t
                 parseAddress | src/orders/address.ts      | t
         priceOrder          | src/orders/pricing.ts       | f
             subtotal        | src/orders/pricing.ts       | f
             applyDiscount   | src/orders/pricing.ts       | f
                 roundCurrency | src/lib/money.ts          | f
         nextId              | src/orders/service.ts       | f
(15 rows)
```

A recursive CTE with `CYCLE` detection, marking the `parseAddress` cycle.

psql: role-based access control in the database

```
cie_demo=> SET ROLE cie_developer;
cie_demo=> SELECT count(*) AS symbols FROM symbol;
 symbols
---------
      57
(1 row)

cie_demo=> SELECT password_hash FROM app_user;
ERROR:  permission denied for table app_user
cie_demo=> INSERT INTO repository (name, url) VALUES ('x', 'test://x');
ERROR:  permission denied for table repository
cie_demo=> CALL review_clone_cluster(1, 1, 'confirmed');
ERROR:  permission denied for procedure review_clone_cluster
cie_demo=> SET ROLE cie_admin;
cie_demo=> BEGIN;
cie_demo=> INSERT INTO repository (name, url) VALUES ('x', 'test://x') RETURNING name, url;
 name |   url
------+----------
 x    | test://x
(1 row)

cie_demo=> ROLLBACK;
```

A developer is refused password hashes, writes and the review procedure; an admin can insert.

psql: pgvector nearest neighbours and HNSW plan

```
cie=# -- V2. Five functions closest in meaning to priceOrder (cosine distance, <=>)
     name      |         path          | cosine_distance | similarity
---------------+-----------------------+-----------------+------------
 applyDiscount | src/orders/pricing.ts |          0.2837 |     0.7163
 subtotal      | src/orders/pricing.ts |          0.3665 |     0.6335
 create        | src/orders/service.ts |          0.3700 |     0.6300
 line_total    | analytics/stats.py    |          0.4255 |     0.5745
 roundCurrency | src/lib/money.ts      |          0.5527 |     0.4473
(5 rows)

cie=# -- V4. EXPLAIN: the k-NN search uses the HNSW index
                                      QUERY PLAN
-----------------------------------------------------------------------------------------------
 Limit
   InitPlan 1 (returns $0)
     ->  Limit
           ->  Index Scan using symbol_embedding_pkey on symbol_embedding symbol_embedding_1
   ->  Index Scan using symbol_embedding_hnsw_idx on symbol_embedding
         Order By: (embedding <=> $0)
(6 rows)
```

Nearest neighbours of `priceOrder`, and a plan that uses the HNSW index.

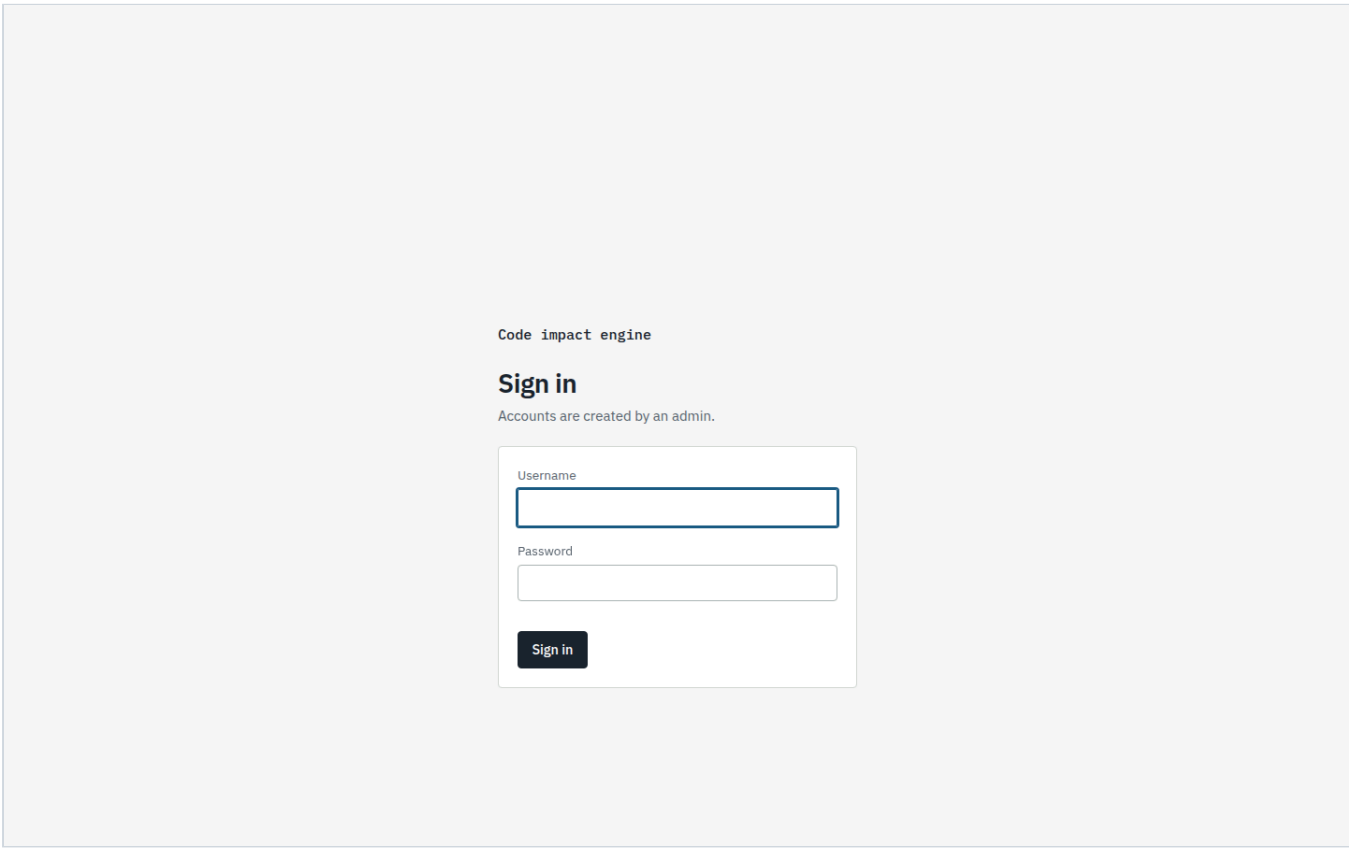
mongosh: update pipeline and validator

```
$ mongosh --quiet cie db/mongo/rud.js
=== U2. updateOne with an aggregation pipeline: cut the tree at depth 8 and recompute
{
  node_count: 135,
  max_depth: 11,
  truncated: false
}
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
{
  node_count: 117,
  max_depth: 8,
  truncated: true
}

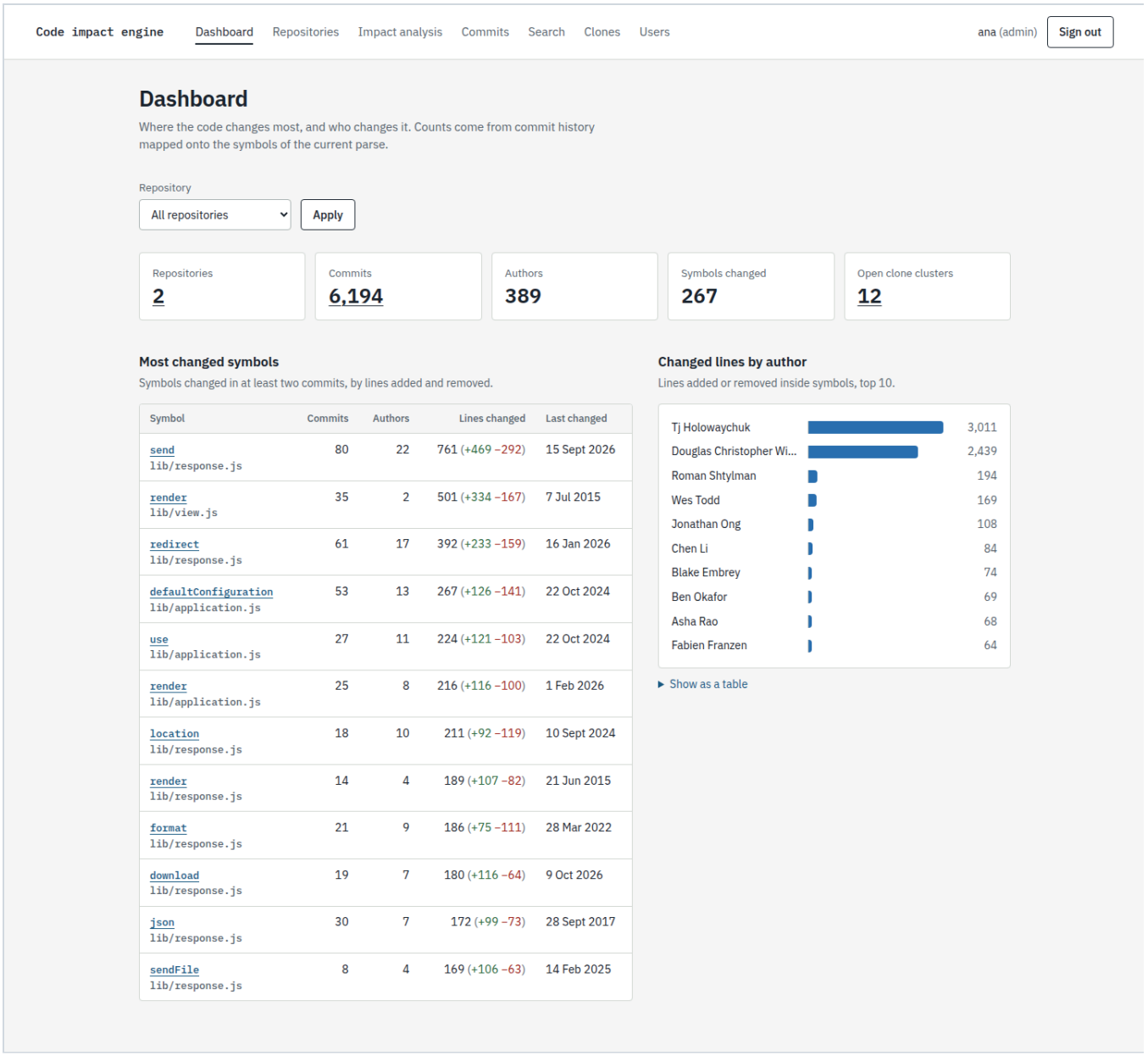
=== U6. The validator and the unique index refuse bad writes
language cobol rejected: code 121, Document failed validation
negative loc rejected: code 121, Document failed validation
duplicate (repo_id, path) rejected: code 11000, E11000 duplicate key error collection: cie_demo.ast index: repo_path_u
```

A pipeline update cuts the tree at depth 8; the validator and unique index refuse bad writes.

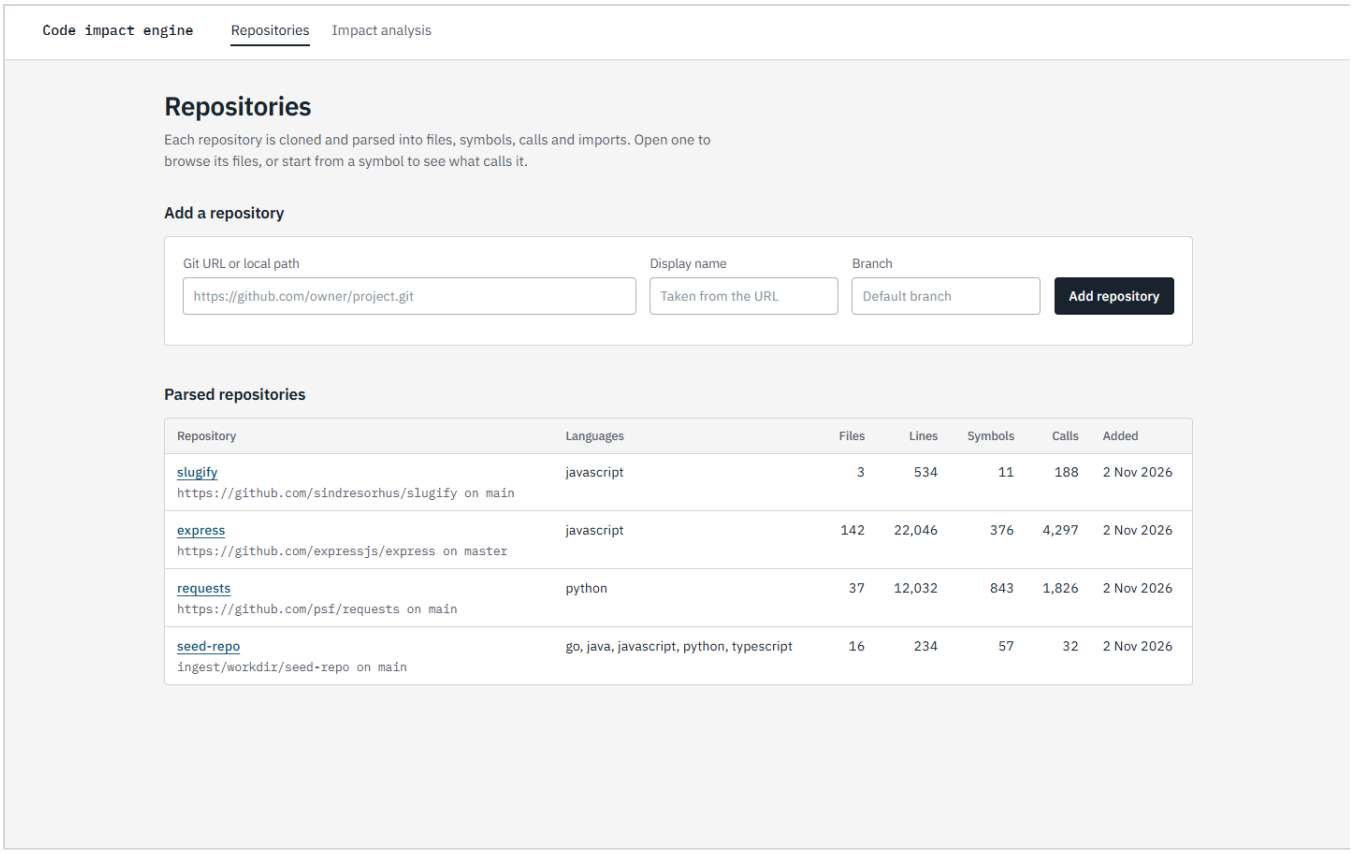
8.3 Application demo



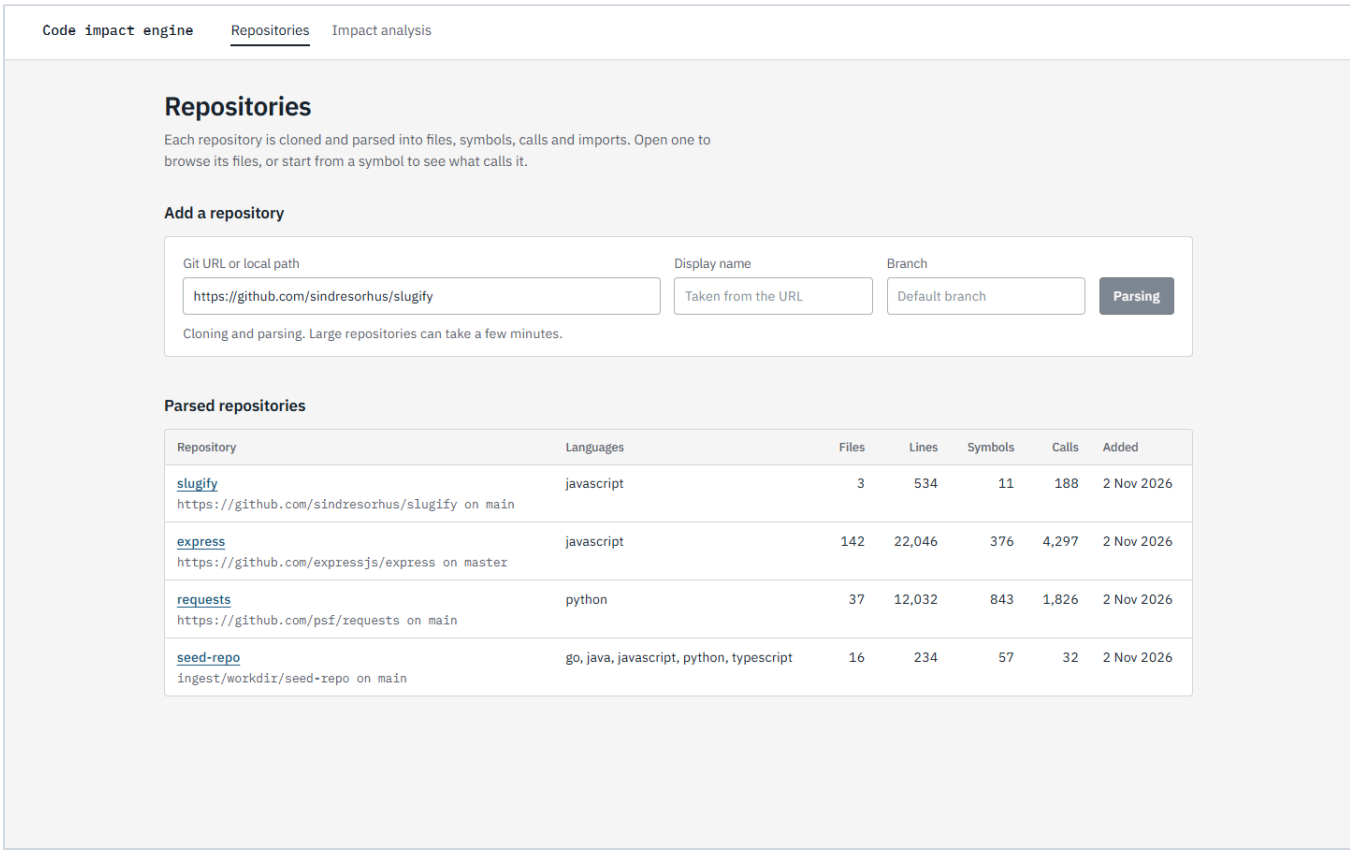
Sign-in page.



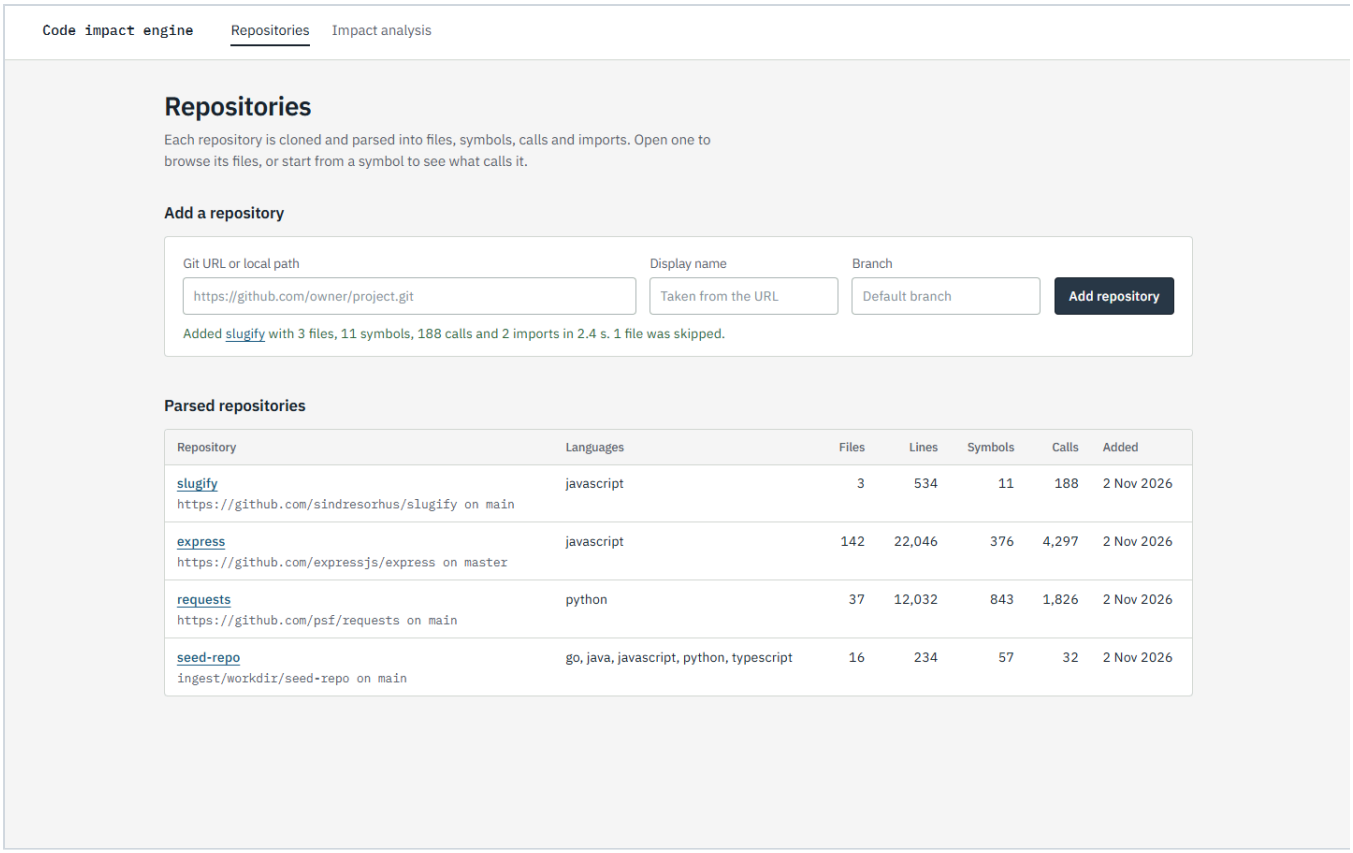
Dashboard: totals, change hotspots and lines changed per author.



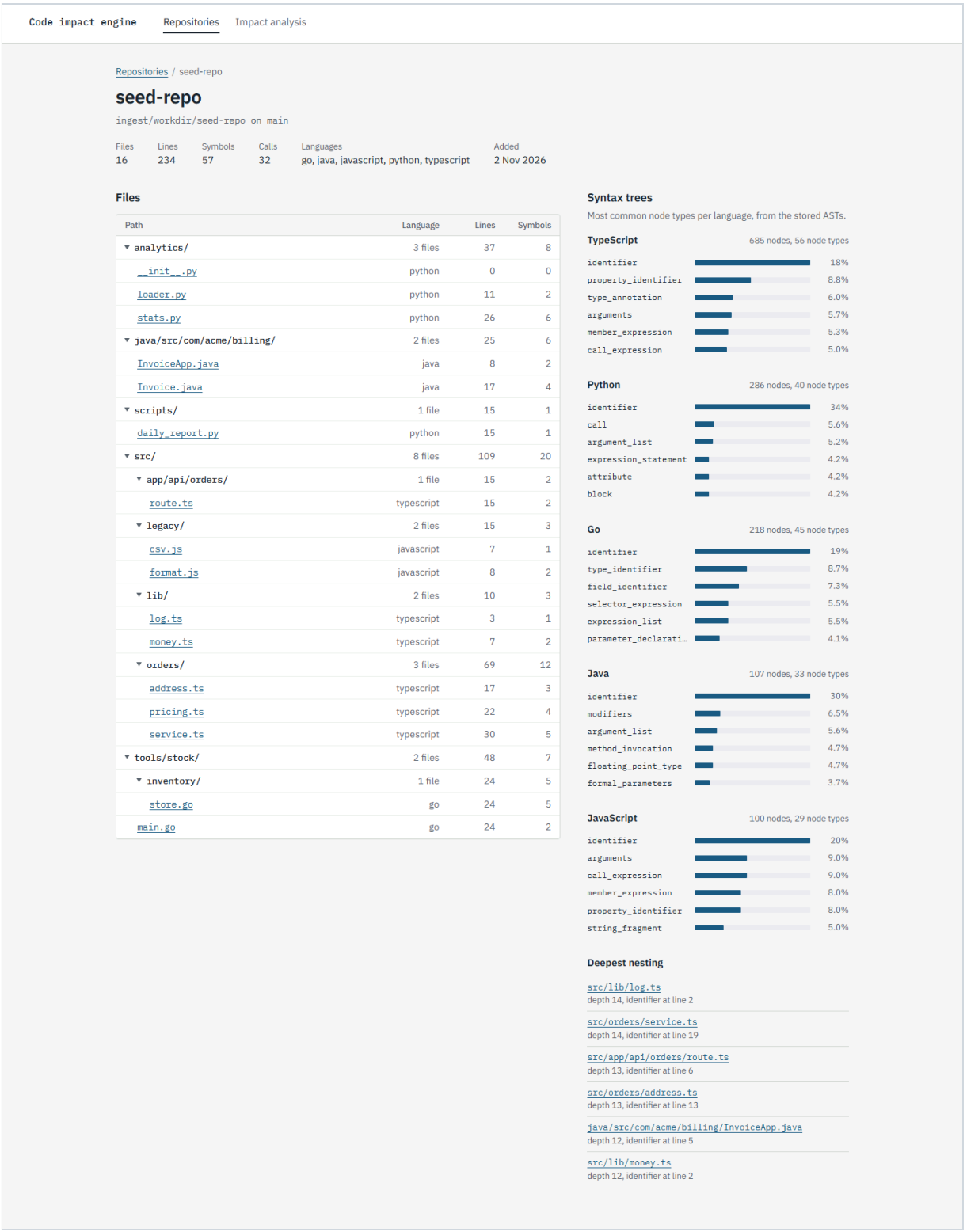
Repository list with the add form for admins.



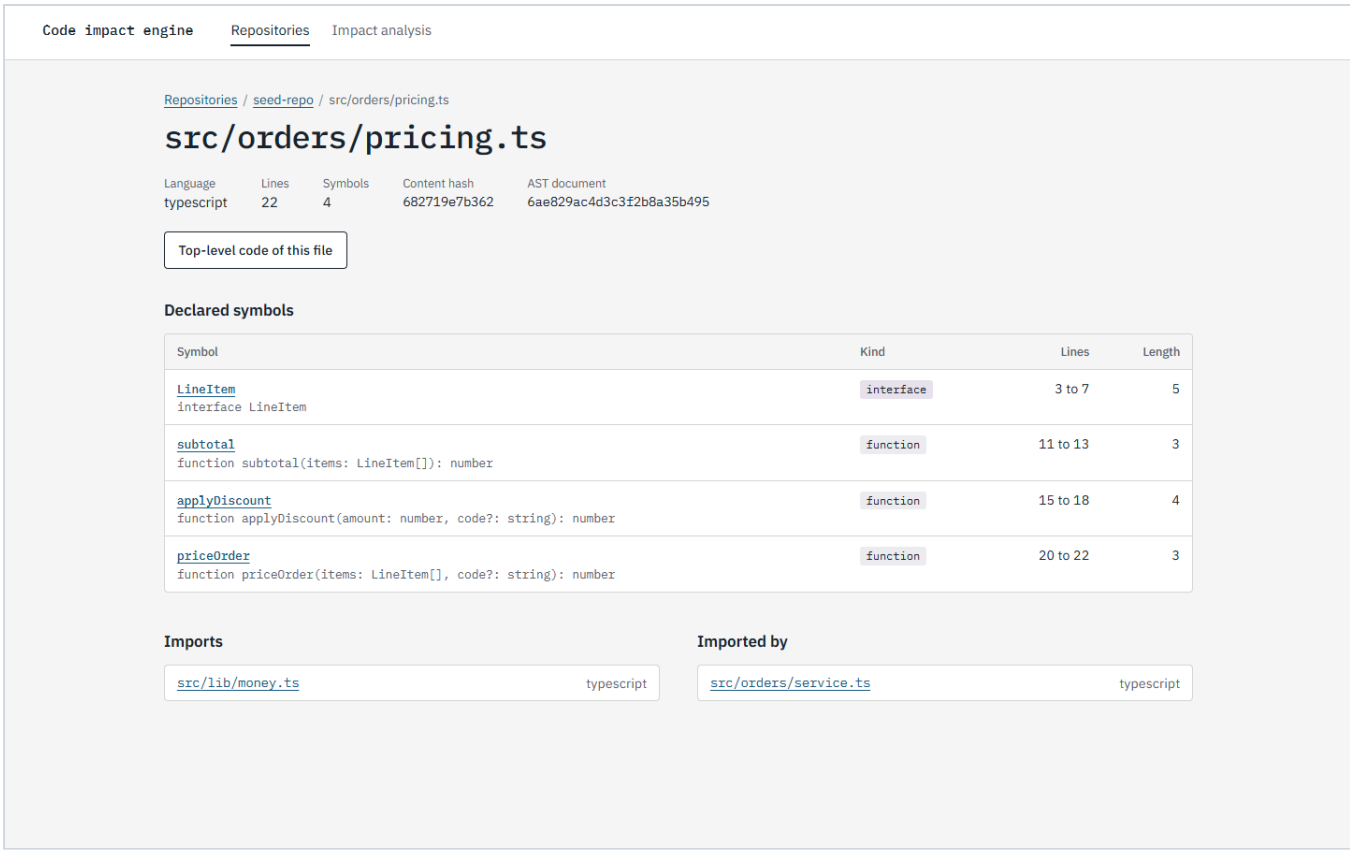
Adding a repository: the app clones and parses it.



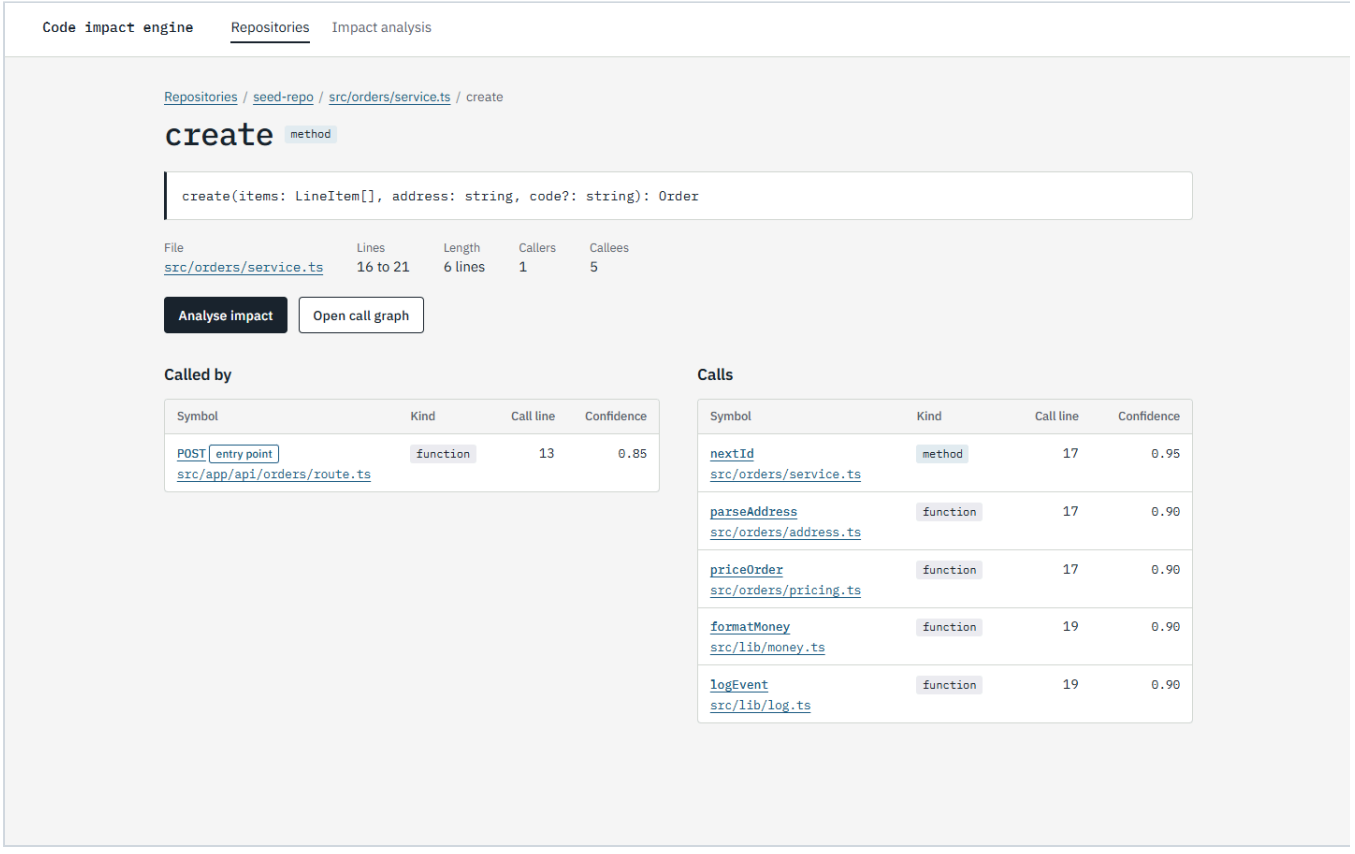
The confirmation after ingestion, with file, symbol, call and import counts.



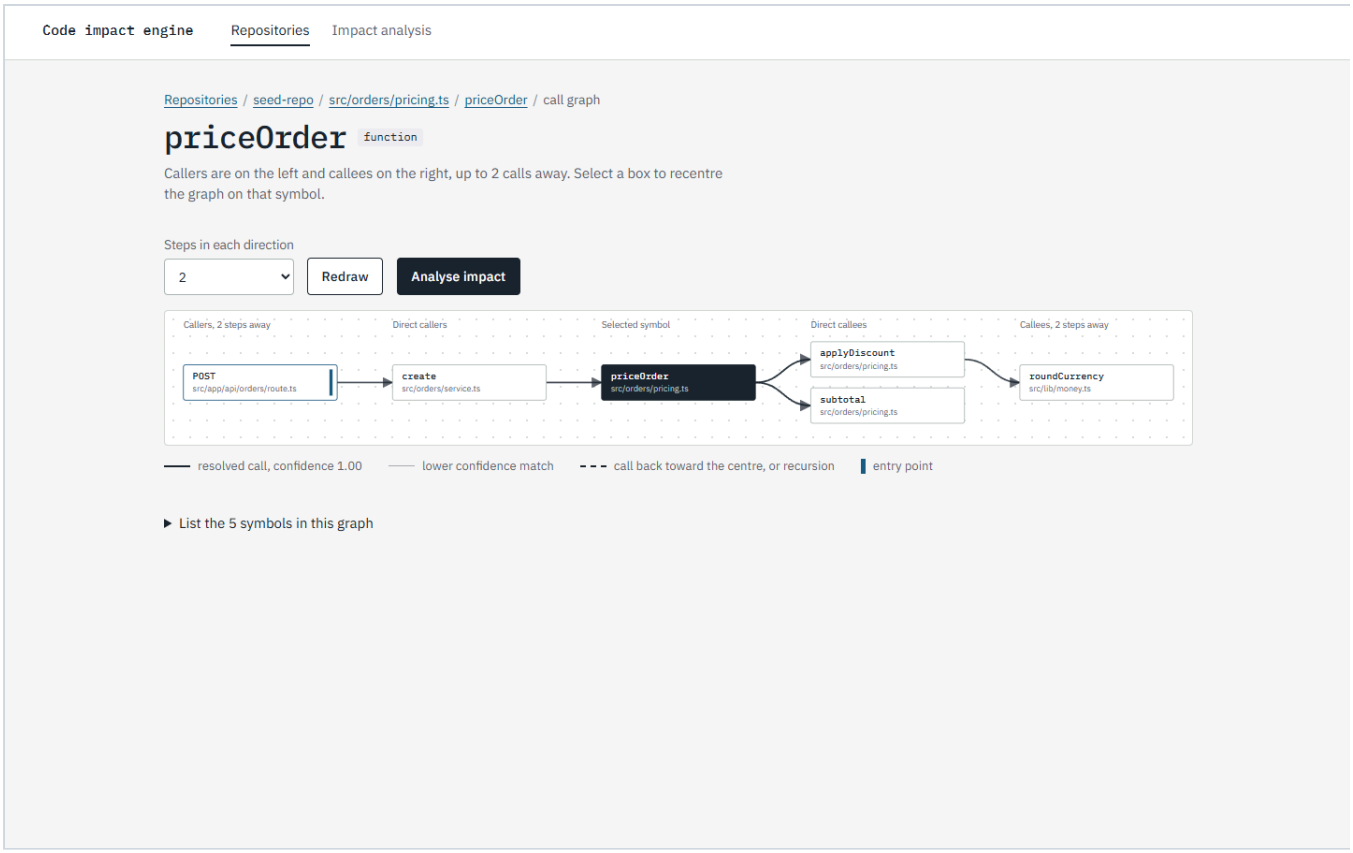
Repository page: file tree and AST statistics from MongoDB.



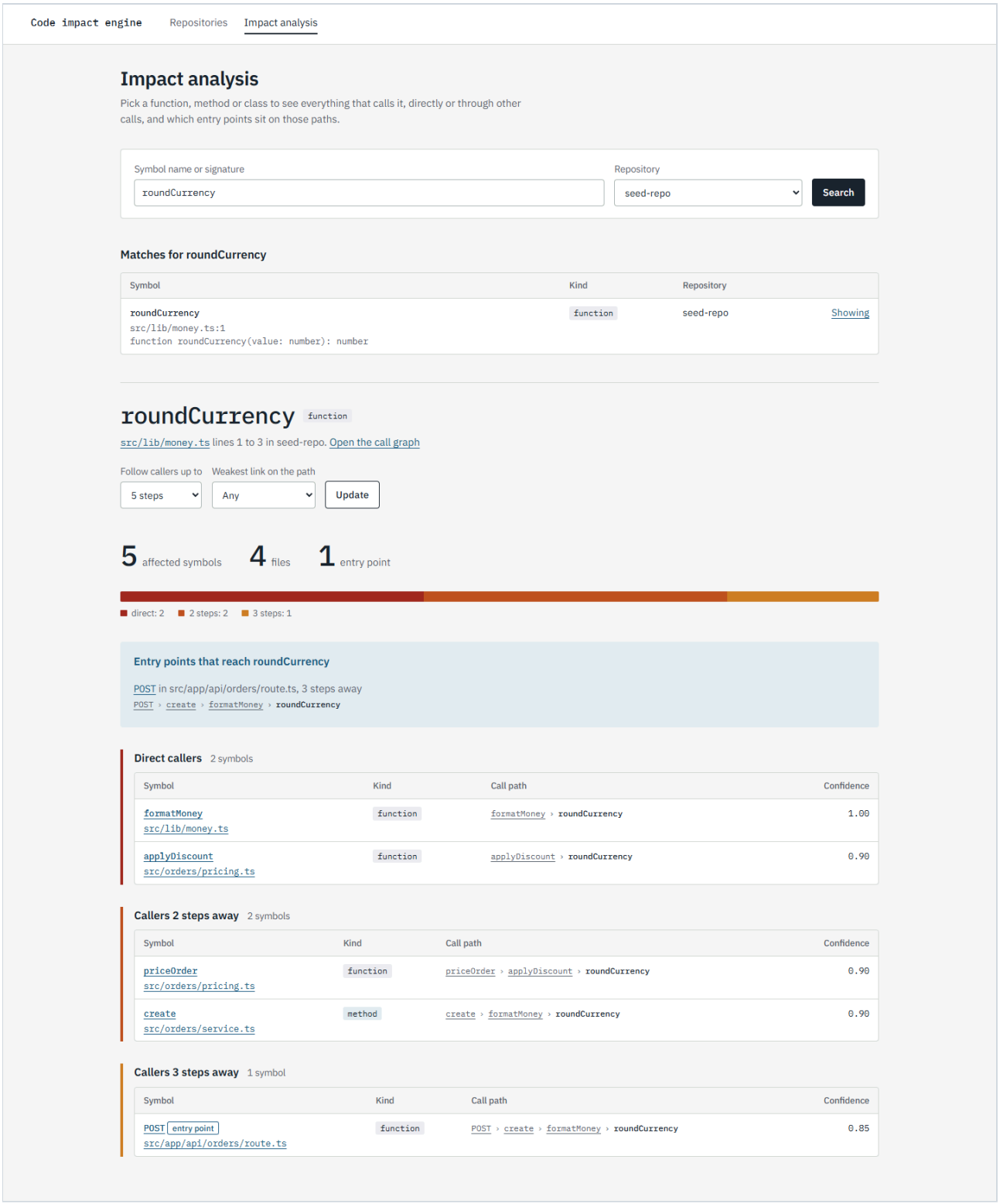
A file and its symbols with their line ranges.



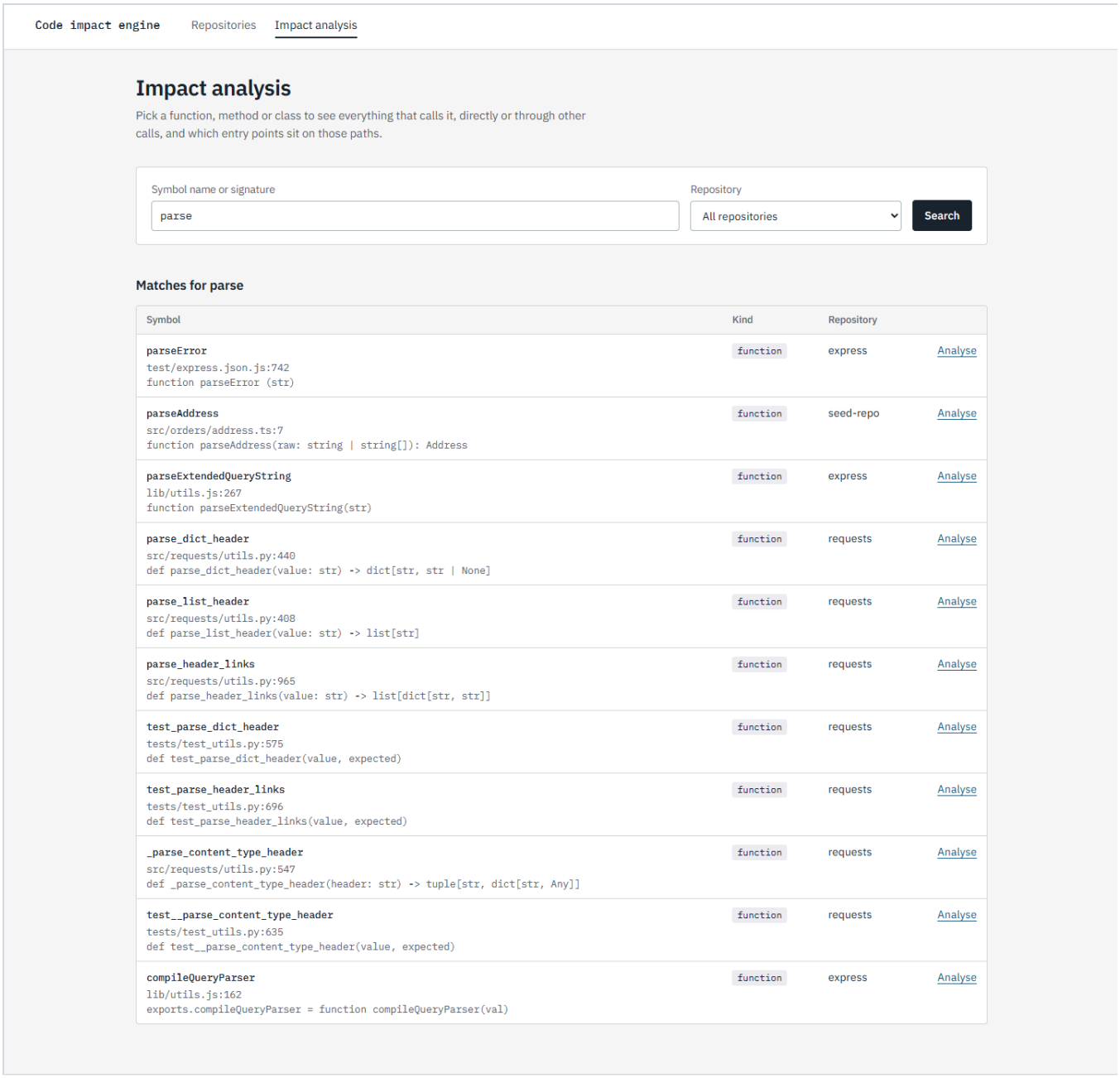
A symbol with its callers and callees.



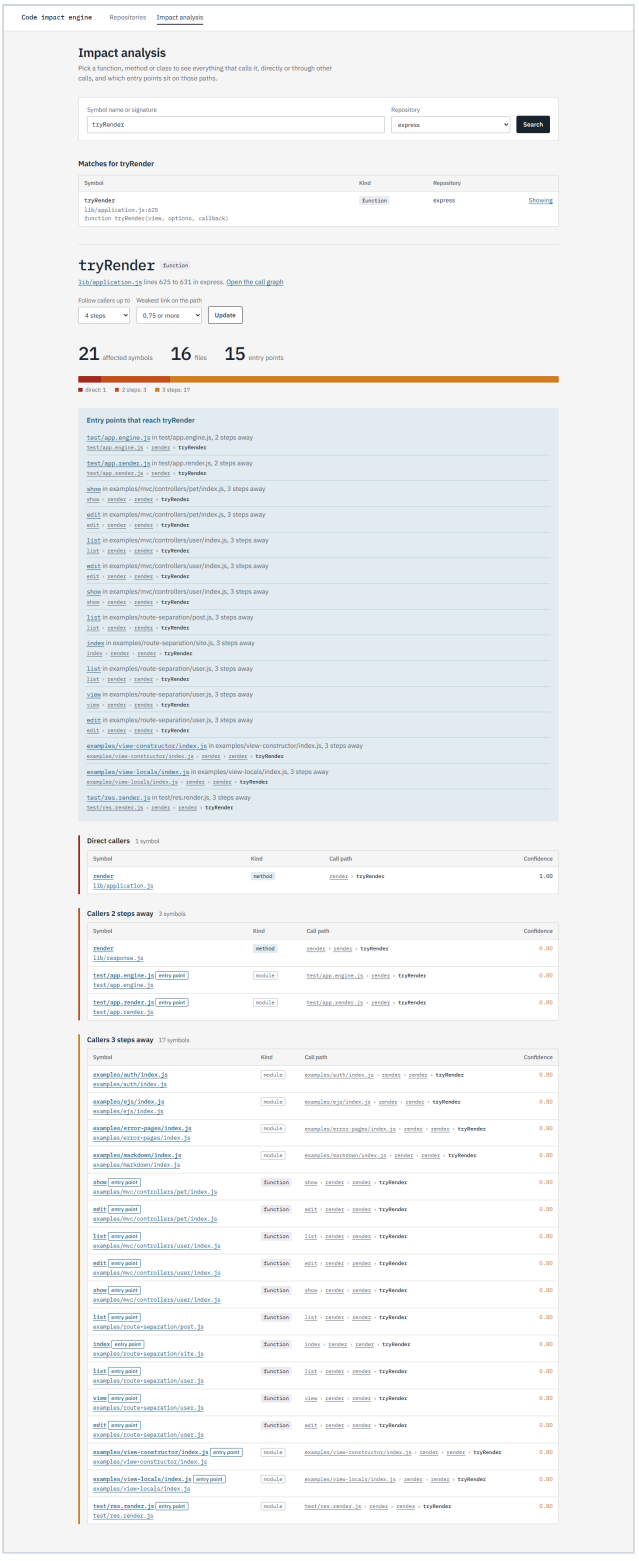
Call graph two levels up and down from a symbol.



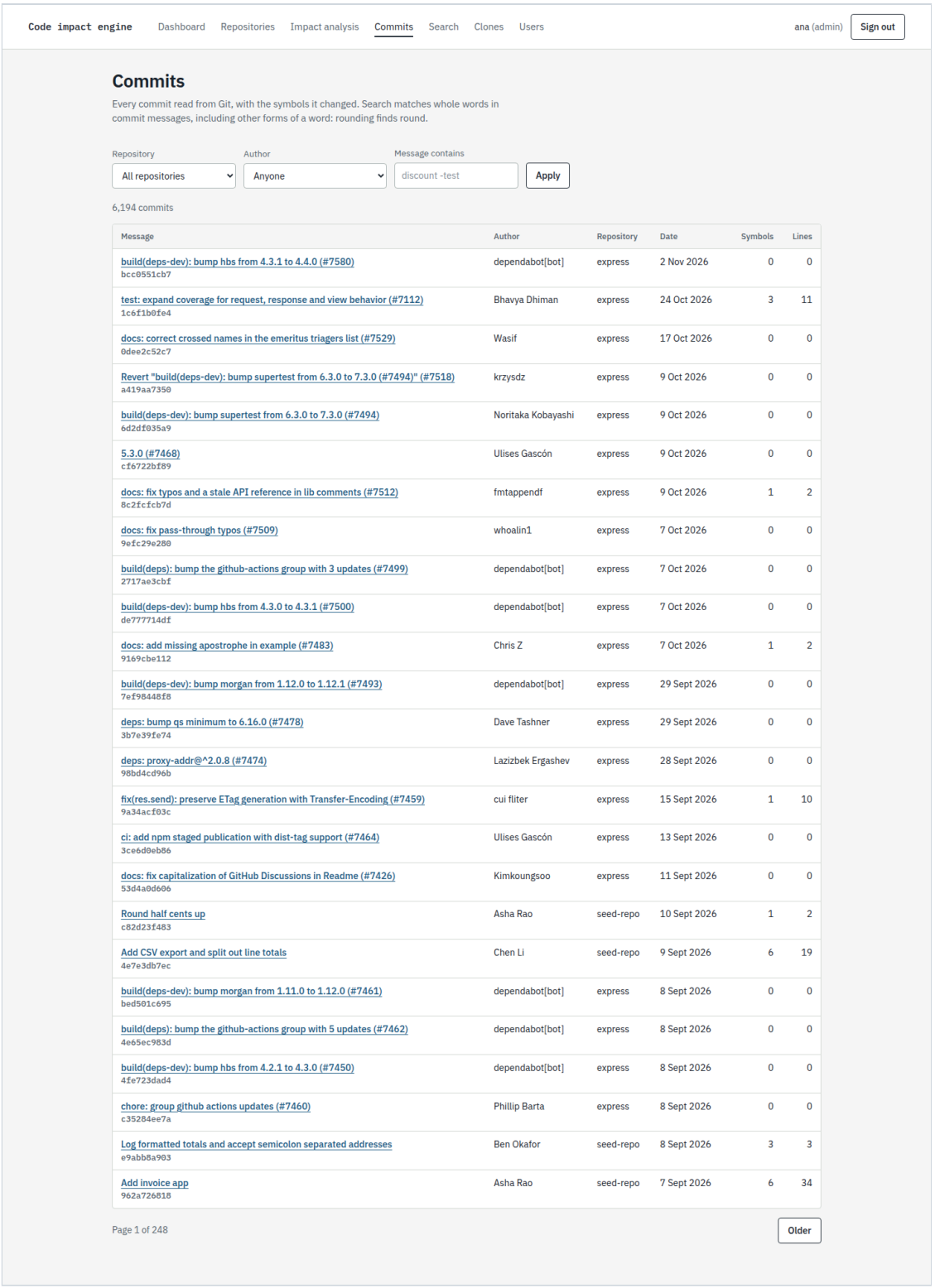
Impact analysis: affected symbols by depth, entry points and confidence.



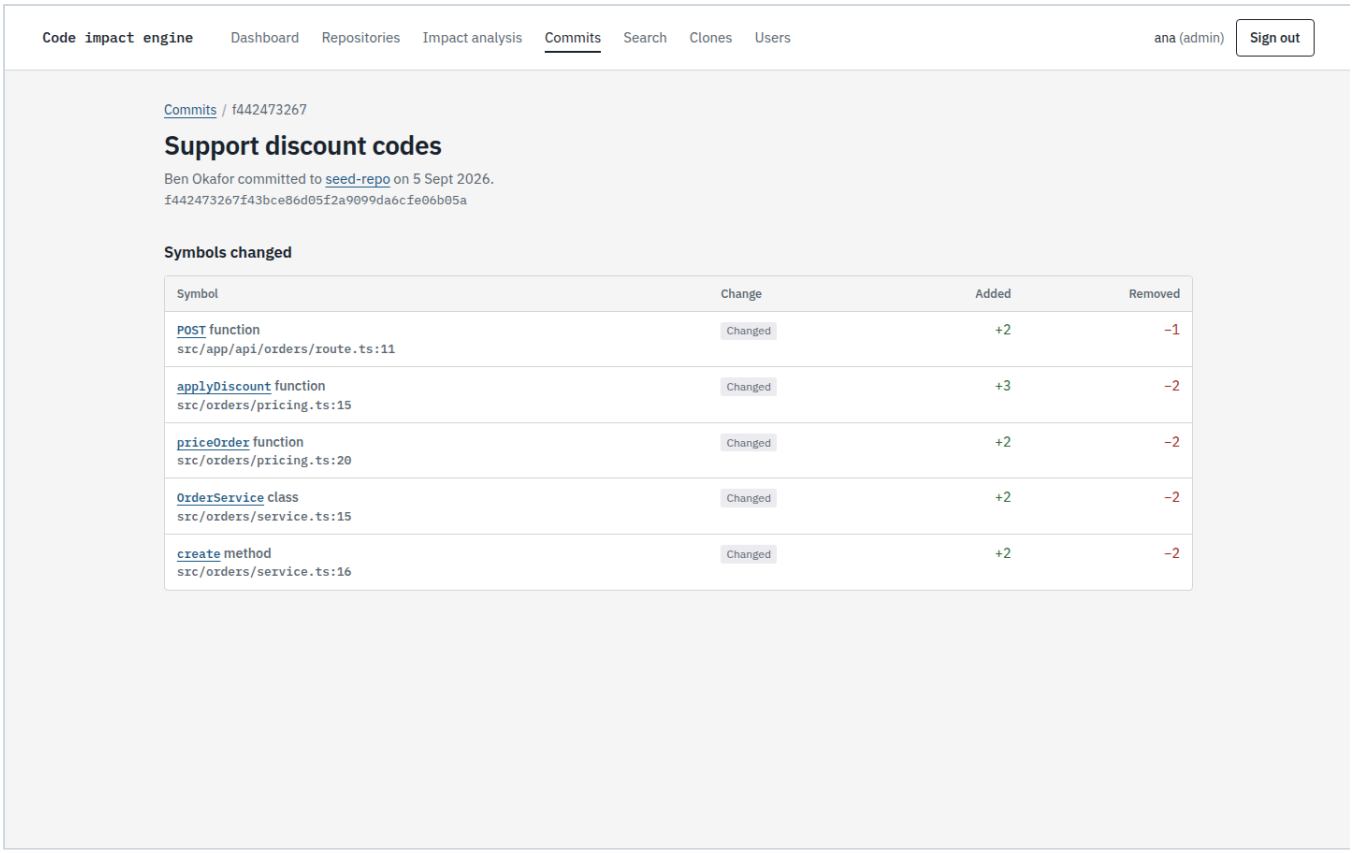
Full-text symbol search.



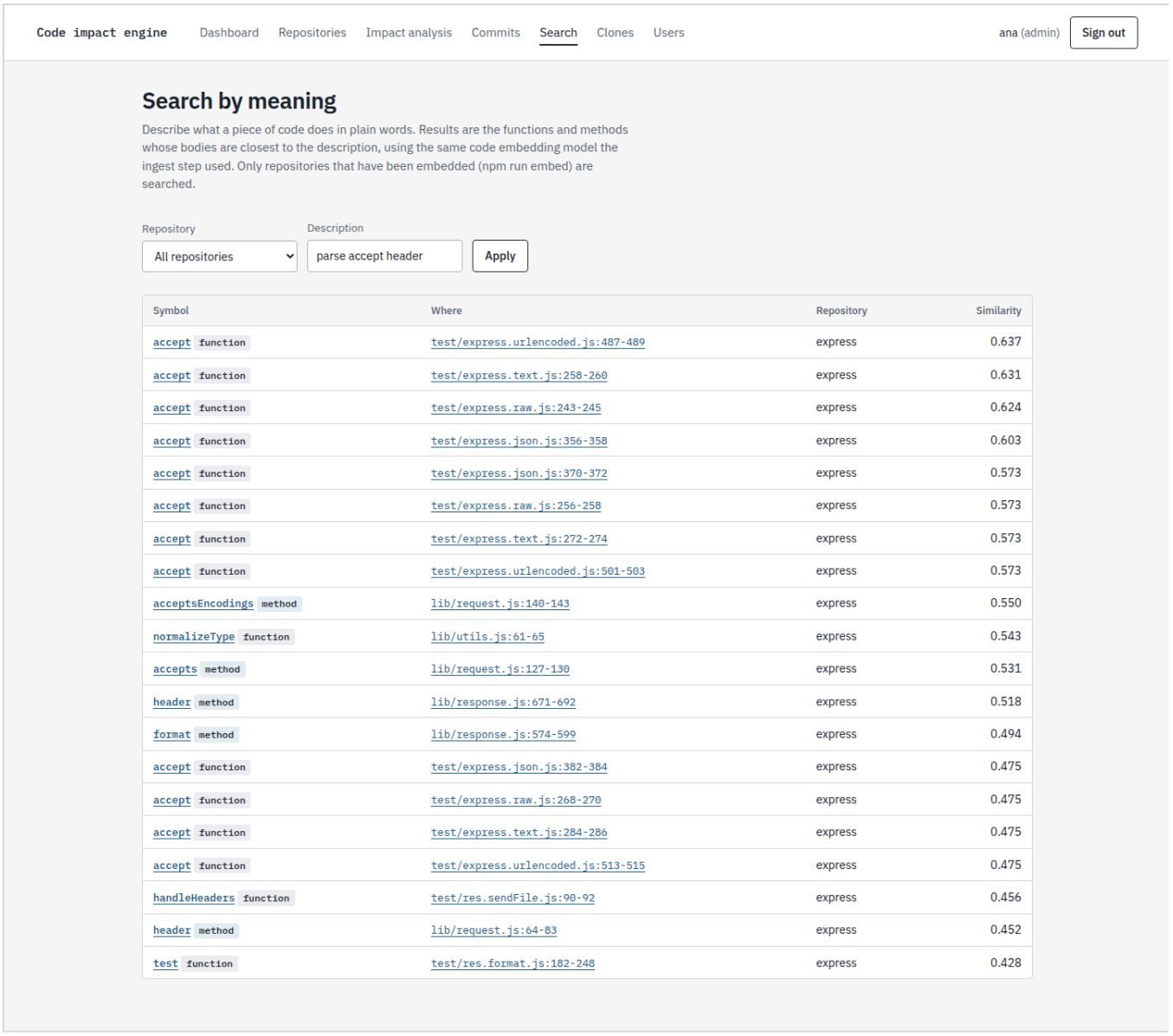
Impact analysis on the `express` repository.



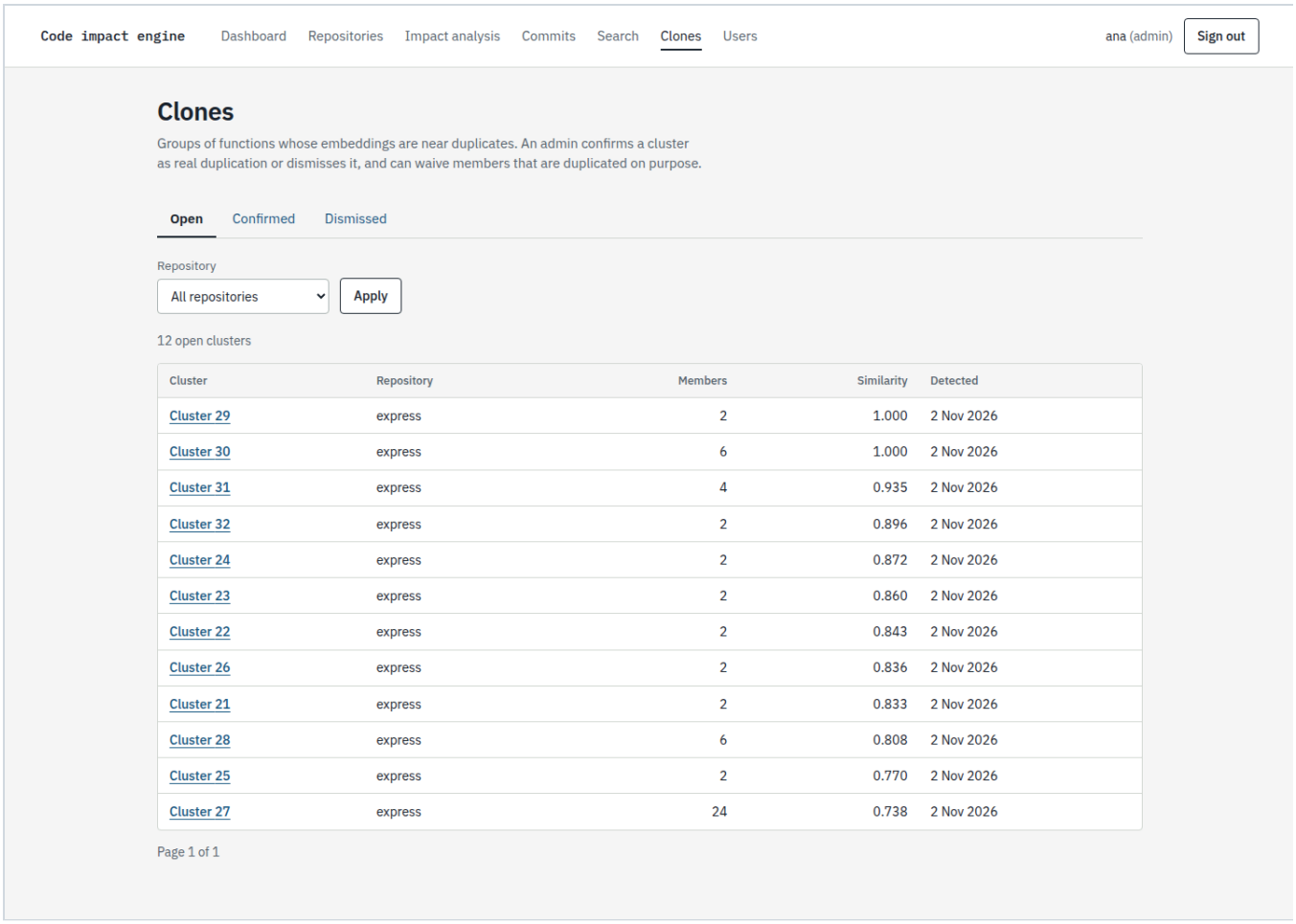
Commit history with repository and author filters and message search.



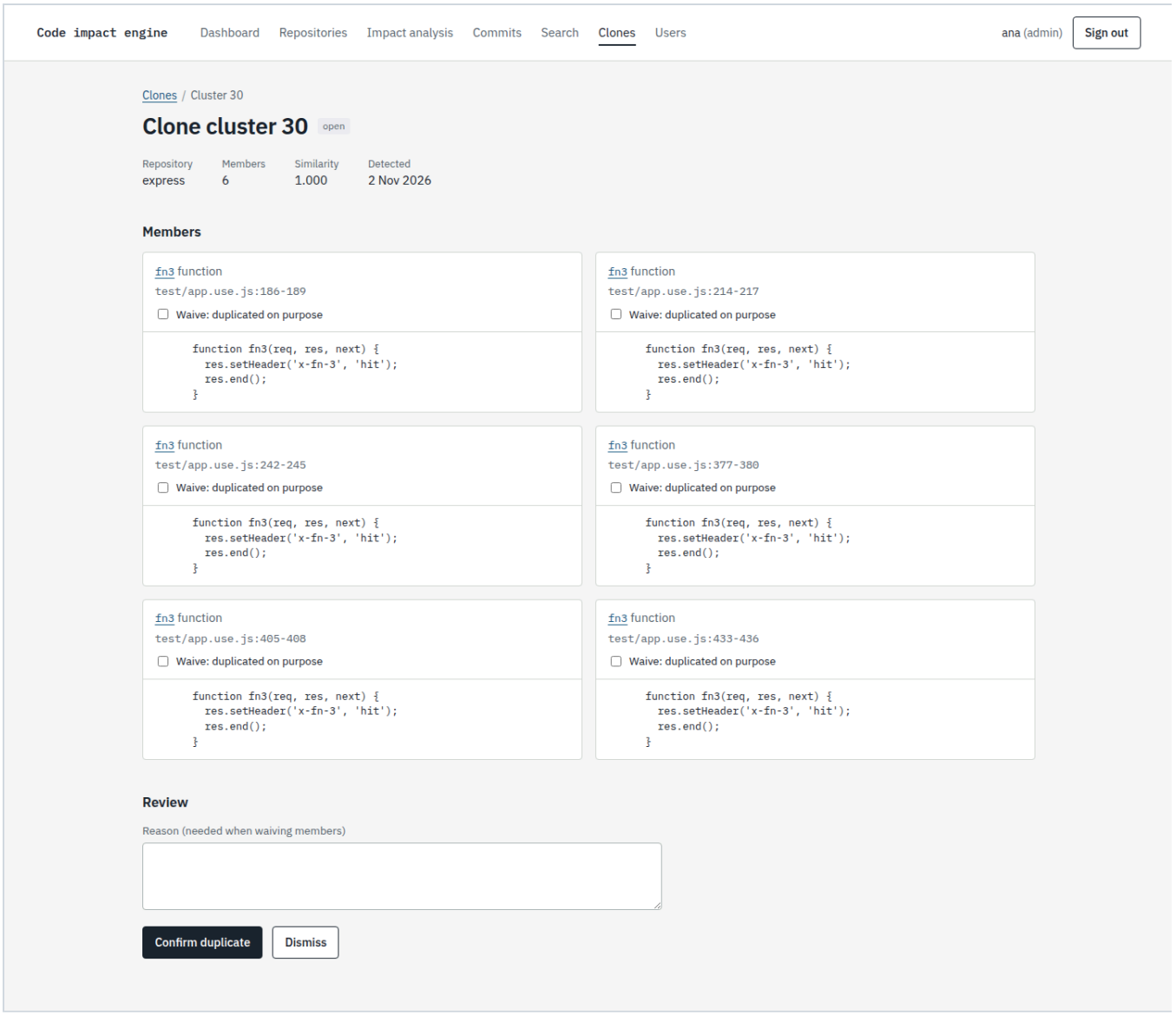
A commit and the symbols it changed.



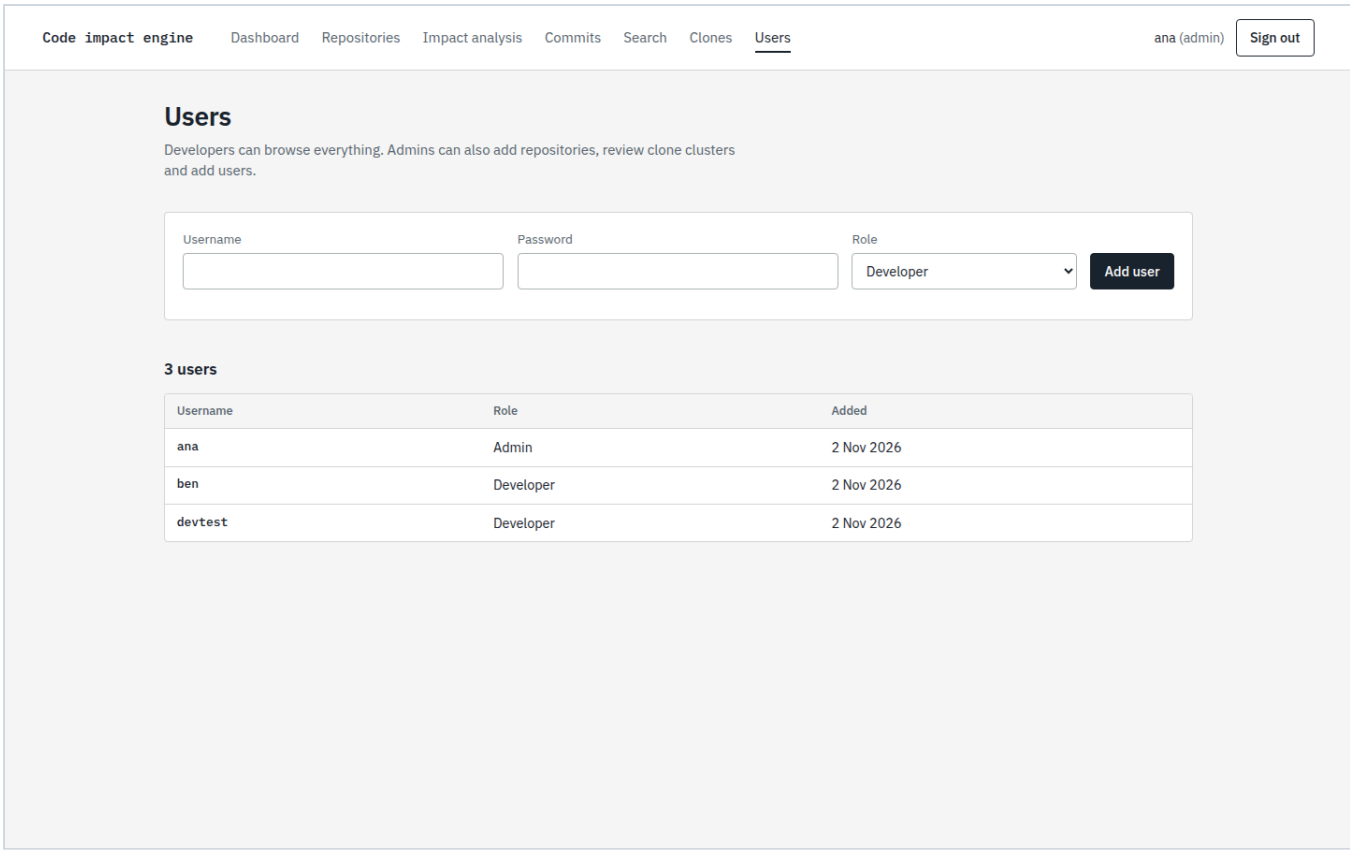
Search by meaning, ranked by cosine similarity.



Clone review queue.



A clone cluster with its members and the review form.



Admin user management.

9. Evaluation rubrics: completion status

#	Deliverable	Status	Evidence
1	ER diagram	Done	ER_Relational_Schema.pdf, section 5.1, DATABASE_DESIGN.md
2	Relational schema with keys, relationships and constraints	Done	ER_Relational_Schema.pdf, section 5.2, mapping steps and data dictionary in DATABASE_DESIGN.md
3	SQL database implementation: DDL with constraints	Done	db/schema, 13 tables with PK, FK, UNIQUE and CHECK constraints
4	NoSQL/vector database creation	Done	db/mongo/init/10_ast.js, ast.schema.json, ast.indexes.json, 31_vectors.sql
5	Technology justification	Done	Section 4.2, DATABASE_DESIGN.md
6	DML operations	Done	dml_operations.sql; ingest writers in `ingest/*/src/store*.ts`
7	Nested, correlated, recursive and join SQL queries	Done	query_types.sql, structure_queries.sql, history_queries.sql
8	Advanced SQL: procedures/functions, triggers, CTEs	Done	db/functions: `impact_of`, `search_symbols`, `review_clone_cluster`, 3 triggers; 3 views
9	NoSQL/vector features and read, update, delete	Done	pipelines.js, rud.js, vector_operations.sql
10	Next.js interface	Done	15 pages under app/; screenshots in section 8.3
11	Backend/API and database integration	Done	17 route handlers under app/api, server actions, lib/pg.ts, lib/mongo.ts; API smoke tests in tests/structure and tests/history
12	Authentication: frontend, two roles, RBAC in the database	Done	Login page, bcrypt hashes, signed session cookie, middleware.ts; roles admin and developer; 40_roles.sql grants and the `clone_cluster_review_guard` trigger
13	Documentation and Git/GitHub	Done	docs/ and the README; one branch per feature, merged through pull requests; `git shortlog -sn` shows each member's commits
14	Originality, individual contribution, modification, viva	Ready for the demo	Ownership by feature in WORK_SPLIT.md; each member presents their own tables, queries and pages

Test status at the time of writing:

Suite	Command	Result
App unit tests	`npm test`	65 of 65 pass
App type check	`npm run typecheck`	No errors
SQL tests	`docker compose exec -T postgres sh /cie-db/tests/run.sh`	8 of 8 files pass
Ingest tests	`cd ingest && npm test`	84 of 84 test cases pass

One known issue: on Windows, the `after` hook of the `BlobReader` suite in `ingest/history/test/blobs.test.ts` fails to delete its temporary Git repository. The `git cat-file` process the reader starts still holds the directory

when the hook runs. The test cases themselves pass, and the directory can be deleted once the test process exits.